



TRABAJO FIN DE GRADO
GRADO EN INTELIGENCIA ARTIFICIAL

Técnicas de inteligencia artificial aplicadas en entornos simulados complejos

Estudiante: Pablo Chantada Saborido
Dirección: Alejandro Rodríguez Arias
Brais Cancela Barizo

A Coruña, julio de 2026.

Para Ainhoa, por ser la bondad en persona.

Agradecimientos

Agradecer a mi Madre y a Heleno, por apoyarme en todo momento y siempre darme todo lo que han podido; a Ainhoa, por entenderme y apoyarme como nadie más lo hace; a Jaime, por siempre estar en vida a pesar de la distancia; a Pepe, por reavivar mi pasión y curiosidad; a mis amigos, por apoyarme durante todo el proceso; y a mis tutores Alex y Brais, por su profesionalidad, guía y enseñanzas durante todo este trabajo.

Resumen

Los videojuegos en 3D ofrecen entornos experimentales excelentes para comparar distintas técnicas de inteligencia artificial en contextos caracterizados por una elevada complejidad perceptiva y temporal. En este trabajo se analizan tres enfoques: la planificación jerárquica de tareas (HTN), el aprendizaje por imitación (IL) y el aprendizaje por refuerzo (RL), empleando el videojuego *Minecraft* como entorno de simulación. La objetivo común elegida consiste en localizar y talar al menos cinco troncos bajo restricciones de mínima omnisciencia; para ello se utiliza una arquitectura multientorno que integra *Node.js* y *Python*. Los resultados muestran que el conocimiento experto es el principal factor diferencial: el HTN alcanza un 100 % de éxito, el IL basado en redes recurrentes un 52 %, y el mejor algoritmo de RL (PPO) un 12 %. Como trabajo futuro, se propone utilizar el HTN como tutor del agente de RL para combinar las fortalezas de ambos paradigmas.

Abstract

Three-dimensional video games provide an ideal testing ground for comparing artificial intelligence techniques in scenarios with high perceptual and temporal complexity. This work compares three paradigms: Hierarchical Task Network (HTN) planning, imitation learning (IL), and reinforcement learning (RL), using the video game *Minecraft* as a simulation environment. The common objective consists of locating and chopping at least five logs under limited perception constraints; to this end, a multi-environment architecture integrating *Node.js* and *Python* is used. Results show that expert knowledge is the main differentiating factor: the HTN achieves a 100 % success rate, the IL based on recurrent networks a 52 %, and the best RL algorithm a 12 %. As future work, it is proposed to use the HTN as a tutor for the RL agent to combine the strengths of both paradigms.

Palabras clave:

- Aprendizaje por refuerzo (RL)
- Aprendizaje por imitación (IL)
- Planificación HTN
- Visión por computador
- Agentes inteligentes
- Minecraft

Keywords:

- Reinforcement Learning (RL)
- Imitation Learning (IL)
- HTN Planning
- Computer Vision
- Intelligent Agents
- Minecraft

Índice general

1	Introducción	1
1.1	Objetivos	2
1.2	Estructura de la Memoria	3
2	Estado del Arte	4
2.1	Técnicas de IA en Videojuegos	4
2.2	Minecraft como Entorno de Investigación	5
2.3	Técnicas de IA aplicadas a Minecraft	5
2.3.1	Aprendizaje por Imitación	5
2.3.2	Aprendizaje por Refuerzo	6
3	Marco Teórico	7
3.1	Planificación Simbólica	7
3.2	Aprendizaje por Imitación	8
3.3	Aprendizaje por Refuerzo	9
4	Metodología y planificación	12
4.1	Enfoque Metodológico	12
4.2	Fases del Proyecto	12
4.2.1	Fases <i>a priori</i>	13
4.2.2	Fases <i>a posteriori</i>	13
4.3	Recursos y limitaciones	14
4.4	Conjuntos de Datos	15
5	Arquitectura del Sistema	17
5.1	Arquitectura General	17
5.1.1	Capa de Simulación	17
5.1.2	Capa de agentes	19

5.1.3	Capa de decisión	19
5.2	Entorno de simulación: Minecraft	20
5.3	Formulación de la tarea	20
5.3.1	Espacio de observaciones	21
5.3.2	Espacio de acciones	22
5.3.3	Condiciones de terminación y éxito	22
5.3.4	Percepción del entorno	22
5.3.5	Reproducibilidad	23
6	Iteración I - Agente HTN	24
6.1	Diseño	24
6.2	Tareas Primitivas	25
6.3	Generación del Dataset de Demostraciones	27
6.4	Resultados de la Iteración	29
6.5	Escalabilidad a tareas de horizonte largo	29
6.5.1	Dependencias entre tareas: precondiciones	30
6.5.2	Alcance del planificador	30
7	Iteración II - Agente de Imitación	31
7.1	Motivación	31
7.2	Entrenamiento sobre el Dataset	32
7.2.1	Fuga de Datos	32
7.2.2	Tratamiento del Desbalanceo	32
7.2.3	Hiperparámetros	33
7.3	Implementación de Solo Estado	33
7.4	Incorporación de la Entrada Visual	34
7.4.1	Cámara como Acciones Discretas y su Limitación	35
7.4.2	Implementación de la Cabeza Dual	36
7.5	Selección del Extractor Visual	37
7.6	Servidor de inferencia	40
7.7	Resultados de la iteración	41
7.7.1	Análisis de resultados	42
7.7.2	Alcance del agente de imitación	43
8	Iteración III - Agente de Refuerzo	44
8.1	Motivación	44
8.2	Formulación del Problema de Refuerzo	45
8.2.1	Espacio de Observación	45

8.2.2	Espacio de Acción	45
8.2.3	Función de Recompensa	46
8.2.4	Condiciones de Terminación	47
8.3	Arquitectura de la Red	47
8.4	Algoritmos de refuerzo	49
8.4.1	Deep Q-Network (DQN)	49
8.4.2	Proximal Policy Optimization (PPO)	49
8.4.3	Soft Actor-Critic (SAC)	50
8.5	Configuración de Entrenamiento	50
8.5.1	Integración con el Entorno	50
8.5.2	Mundo de Entrenamiento	50
8.5.3	Hiperparámetros	50
8.5.4	Selección del <i>Checkpoint</i>	51
8.6	Resultados de la Iteración	53
8.6.1	Curvas de aprendizaje y comparativa global	53
8.6.2	Estabilidad del entrenamiento	55
8.6.3	Distribución de acciones	56
8.6.4	Discusión	57
9	Discusión de Resultados	58
9.1	Metodología de Evaluación	58
9.2	Resultados por Familia	59
9.2.1	Agente Experto	60
9.2.2	Aprendizaje por Imitación	60
9.2.3	Aprendizaje por Refuerzo	60
9.3	Comparativa Global	60
10	Conclusiones	62
10.1	Trabajo Futuro	63
A	Jerarquía de un Pico de Hierro	65
B	Frame del dataset de demostraciones	66
C	Árbol de Tareas para la Obtención de un Pico de Hierro	68
D	Variantes de imitación	70
	Lista de acrónimos	71

Glosario	73
Bibliografía	77

Índice de figuras

2.1	Arquitectura del agente Voyager	6
3.1	Ilustración del proceso básico de aprendizaje por refuerzo	9
3.2	On-policy vs off-policy	11
5.1	Arquitectura general del sistema	18
5.2	Jerarquía de clases de la capa de agentes	19
5.3	Entorno espacial de <i>Minecraft</i>	20
6.1	Árbol de descomposición de la tarea de woodcutting	24
6.2	Bucle de ejecución de una tarea	27
6.3	Distribución de clases de acción en el dataset HTN	28
6.4	Resolución recursiva de precondiciones	30
7.1	Arquitectura de la variante solo-estado	34
7.2	Arquitectura de cabeza única (cámara discreta)	35
7.3	Arquitectura de doble cabeza del agente de imitación	36
7.4	Curvas de entrenamiento de los extractores visuales	38
7.5	Matriz de confusión del agente de imitación	39
7.6	Matriz de confusión binaria (atacar vs movimiento)	40
7.7	Bucle de inferencia del agente de imitación	41
7.8	Distribución de acciones del agente de imitación por técnica	42
8.1	Arquitectura de red del agente de refuerzo	48
8.2	Bucle de entrenamiento del agente de refuerzo	51
8.3	Tasa de éxito (media móvil, ventana 20)	54
8.4	Recompensa por episodio (media móvil, ventana 20)	54
8.5	Señales de estabilidad de SAC y DQN	55
8.6	Distribución de acciones por algoritmo de refuerzo	56

8.7	Distribución de acciones por algoritmo de refuerzo con el mismo espacio de acciones que el agente de imitación.	57
9.1	Tasa de éxito por técnica	59
9.2	Tasa de éxito según el umbral de troncos	61
A.1	Jerarquía de dependencias para obtener un pico de hierro en <i>Minecraft</i>	65
B.1	Imagen del visor en primera persona	66
C.1	Árbol de descomposición general del agente simbólico	68
C.2	Tarea HTN: Fase Madera	68
C.3	Tarea HTN: Fase Piedra	69
C.4	Tarea HTN: Fase Hierro	69
D.1	Curvas de entrenamiento de las variantes de imitación	70

Índice de tablas

4.1	Equipos empleados en el desarrollo del proyecto.	14
4.2	Conjuntos de datos analizados	15
5.1	Elementos del vector de estado	21
5.2	Espacio de acciones discreto	22
6.1	Primitivas del agente simbólico para la tarea de woodcutting	25
6.2	Estructura de los <i>frames</i> del dataset.	28
6.3	Evaluación de la técnica simbólica	29
7.1	Configuración de entrenamiento del agente de imitación	33
7.2	Cabeza única frente a cabeza dual	36
7.3	Comparación de extractores visuales	38
7.4	Comparación de extractores visuales	39
7.5	Evaluación de las técnicas de imitación	42
8.1	Términos de la función de recompensa	47
8.2	Configuración de entrenamiento del agente de refuerzo	52
8.3	Comparativa de algoritmos de refuerzo en entrenamiento	53
8.4	Evaluación de las técnicas de refuerzo	53
9.1	Evaluación de las técnicas	59

Introducción

Los entornos virtuales en *3D* han resultado ser buenos escenarios para la investigación de algoritmos de *Inteligencia Artificial (IA)*. A diferencia de entornos de juegos clásicos como el ajedrez, los entornos virtuales en *3D* ofrecen percepción visual en diferentes perspectivas (primera persona, tercera persona, etc.), espacios de acción continuos o de alta dimensionalidad y características temporales complejas, aumentando mucho más su dificultad y acercándolos más al funcionamiento del mundo real [1, 2]. Plataformas como Atari [3] o VizDoom [4] han sido referentes clave para el uso de aprendizaje por refuerzo y planificación autónoma en los videojuegos.

Dentro de esta familia de entornos virtuales, *Minecraft* ocupa un lugar singular. Se trata de un videojuego *3D* de mundo abierto en primera persona centrado en la recolección de recursos, la construcción de estructuras, la fabricación de objetos y la supervivencia [5]. Se caracteriza por sus mundos, generados de forma procedimental y formados por bloques que el jugador puede modificar, lo que genera un espacio de acciones grande y variado. A diferencia de otros entornos de referencia, *Minecraft* carece de un objetivo único predefinido, lo que obliga a los agentes a planificar horizontes temporales largos con recompensas dispersas y, a menudo, a trabajar en situaciones nunca vistas durante el entrenamiento.

El desafío más complejo que presenta el juego consiste en derrotar al *Ender Dragon*, un jefe final que requiere la planificación y ejecución de una gran red de tareas anidadas entre sí. Tarea que aún no ha sido completada usando únicamente agentes autónomos, sin ninguna intervención humana.

A nivel de progreso, todos los objetivos que plantee el jugador pueden alcanzarse de varias formas y empleando diferentes herramientas. A partir de esta premisa, los jugadores pueden fabricar materiales y objetos específicos que requieren la recolección previa de una variedad de recursos, lo que da lugar a un complejo sistema de creación organizado como un árbol jerárquico (véase el Apéndice A).

Como ejemplo, si el jugador quisiera un pico de hierro debería: recolectar primero madera

rompiendo un árbol, luego fabricar una mesa de trabajo para fabricar nuevos objetos; dentro de ella, fabricar un pico de madera para recolectar piedra y, a continuación, un pico de piedra con el que extraer carbón y mineral de hierro; después, construir un horno para fundir el mineral y obtener lingotes de hierro para, finalmente, fabricar el pico de hierro. Además, el jugador debe encontrar en el mundo todos los recursos necesarios.

Esta tarea, que puede parecer sencilla para un jugador humano, supone un gran desafío para los algoritmos de IA, ya que tanto el entorno como el espacio de acciones son muy grandes y complejos, y presentan además un fuerte componente temporal difícil de capturar.

Ante este desafío se pueden aplicar paradigmas de IA muy diferentes entre sí, desde la planificación simbólica basada en conocimiento experto hasta el aprendizaje a partir de imitación o usando la propia interacción con el entorno. Este trabajo, por tanto, compara estas tres familias: planificación jerárquica de tareas (*Hierarchical Task Network* (Red de Tareas Jerárquicas) (HTN)), aprendizaje por imitación (*Imitation Learning* (Aprendizaje por Imitación) (IL)) y aprendizaje por refuerzo (*Reinforcement Learning* (Aprendizaje por Refuerzo) (RL)), sobre una misma tarea en *Minecraft* y bajo idénticas condiciones de evaluación, con el fin de analizar las fortalezas y limitaciones de cada enfoque y la relación entre conocimiento experto, datos y coste de cómputo.

1.1 Objetivos

El trabajo plantea las siguientes metas finales:

- Diseñar e implementar agentes autónomos basados en distintos paradigmas de IA. Específicamente HTN, IL y RL, enfocados en la resolución de tareas en *Minecraft*.
- Desarrollar, en caso necesario, conjuntos de datos específicos que permitan el entrenamiento de los modelos seleccionados.
- Definir métricas adecuadas para la evaluación del rendimiento de los agentes en el entorno.
- Analizar y comparar el rendimiento, eficacia y limitaciones de cada paradigma.
- Combinar sistemas de forma eficiente, utilizando diferentes lenguajes de programación para el desarrollo de modelos y el control de agentes, así como la comunicación entre distintos entornos de desarrollo.

1.2 Estructura de la Memoria

La organización de este trabajo es la siguiente: tras el Capítulo 1 de introducción, se hace una revisión del estado del arte en el Capítulo 2, enfocado en técnicas de IA aplicadas a videojuegos y, en particular, *Minecraft*. A continuación, el Capítulo 3 introduce los fundamentos teóricos de las tres familias de técnicas comparadas (HTN, IL y RL), mientras que el Capítulo 4 detalla la metodología de desarrollo, las fases del proyecto, su planificación y los conjuntos de datos empleados. El Capítulo 5 describe la arquitectura del sistema común a todas las iteraciones. Sobre esta arquitectura se construyen las tres iteraciones que forman el núcleo del trabajo: el Capítulo 6 introduce al agente HTN, que actúa de experto; el Capítulo 7, el agente IL que aprende clonando a dicho experto; y el Capítulo 8, el agente RL que aprende desde cero por interacción con el entorno. Tras esto, en el Capítulo 9 se comparan las tres técnicas bajo una métrica común y se analiza la relación entre conocimiento experto, datos y coste de cómputo. Por último, se dedica el Capítulo 10 a exponer las conclusiones y las posibles líneas futuras de investigación obtenidas del análisis realizado en el trabajo.

Estado del Arte

EN este capítulo se ofrece una explicación del estado del arte sobre el uso de videojuegos como entorno de investigación en IA, enfocándose especialmente en *Minecraft*. Se describen los trabajos más relevantes en el ámbito de los videojuegos, haciendo especial énfasis en aquellos que utilizan *Minecraft* como entorno de experimentación.

Los juegos han sido un buen campo de prueba para el desarrollo de la IA, gracias a su capacidad para simular entornos complejos e interactivos, y a la posibilidad de medir el rendimiento de los agentes de manera objetiva. Desde juegos clásicos con espacios de acción limitados, como el ajedrez [6] o el Go [1], que pese a disponer de un espacio de acciones reducido generan un espacio de estados prácticamente infinito, hasta juegos de estrategia en tiempo real con espacios de acciones enormes, como *StarCraft* [2] o *Dota 2* [7], con un número de estados casi imposible de explorar.

La temática común entre estos artículos es el uso del RL (clásico o profundo) para entrenar agentes que puedan competir a nivel humano o superarlo. A diferencia de los juegos de mesa mencionados anteriormente, que se basan en turnos, ejemplos como *Space Invaders* o *Pong* requieren una percepción visual y una respuesta rápida para interactuar con el entorno de forma óptima, lo que los convierte en un campo de prueba excelente para el *Deep Reinforcement Learning* (Aprendizaje por Refuerzo Profundo) (DRL), con las *Deep Q-Network* (DQN) [3] como una de las primeras aproximaciones significativas en mezclar refuerzo clásico con aprendizaje profundo.

2.1 Técnicas de IA en Videojuegos

Entre los enfoques aplicados a videojuegos, el DRL ha producido resultados destacables. Las redes DQN alcanzaron un rendimiento comparable al de jugadores humanos profesionales en un conjunto de 49 juegos de Atari [3], *AlphaStar* llegó a nivel Gran Maestro en *StarCraft II* [2] y *OpenAI Five* superó a equipos profesionales en *Dota 2* [7]. Su principal li-

mitación es la necesidad de una gran cantidad de interacciones con el entorno y el diseño de una función de recompensa correcta.

Por otra parte, las HTN han demostrado ser competitivas en juegos de estrategia en tiempo real como *StarCraft* [8], produciendo comportamientos deterministas e interpretables sin necesidad de entrenamiento, aunque a costa de requerir conocimiento del dominio codificado manualmente.

Sin embargo, con la llegada de los *Transformer* [9], se han empezado a aplicar estas técnicas a la investigación en videojuegos con resultados prometedores [10, 11, 12, 13]. Estas aproximaciones difieren entre sí: algunas emplean un *Large Language Model* (Modelo de Lenguaje Grande) (LLM) para generar directamente las acciones o el código que controla al agente en lugar de una red neuronal entrenada por refuerzo. En el caso de *Voyager* [11], el LLM genera código *JavaScript* para controlar a un agente en *Minecraft* en función de su entorno. Otras, como *Oasis* [13], emplean el *Transformer* como modelo generativo para construir un clon interactivo de *Minecraft*.

2.2 Minecraft como Entorno de Investigación

Actualmente existen diversos trabajos relacionados con el uso de *Minecraft* como entorno de investigación en IA, como el proyecto *MineRL* [5], que proporciona un conjunto de datos y un entorno de simulación para entrenar agentes RL en *Minecraft*.

En esta misma línea, *MineDojo* [12] amplía el enfoque con un conjunto masivo de tareas abiertas y una extensa base de conocimiento multimodal extraída de Internet (vídeos, wiki y foros).

En el ámbito de los LLM, *Mindcraft* [14] es un *Multi-Agent System* (Sistema Multiagente) (MAS) basado en LLM para el razonamiento colaborativo, en el que los agentes generan código *JavaScript* para actuar en *Minecraft*, de forma similar al enfoque empleado por *Voyager* [11], cuya arquitectura se muestra en la Figura 2.1 (página 6). El objetivo a largo plazo de estos sistemas es completar el juego de forma totalmente autónoma, sin intervención humana.

2.3 Técnicas de IA aplicadas a Minecraft

2.3.1 Aprendizaje por Imitación

El referente de IL en *Minecraft* es *Video PreTraining* (VPT) [10], que preentrena una política sobre 70 000 horas de vídeo sin etiquetar, logrando comportamientos complejos por imitación. Su limitación principal es el desajuste entre los estados del experto y los que explora el aprendiz (*distribution shift*). Para mitigarlo, *Dataset Aggregation* (Dagger) [15] emplea



Figura 2.1: Arquitectura de *Voyager* [11], un sistema basado en LLM para controlar un agente en *Minecraft*. El LLM genera código JavaScript para controlar al agente en función de su entorno y construye de forma incremental una biblioteca de habilidades reutilizables.

al experto sobre los estados recorridos para corregir los errores cometidos por el aprendiz, aunque el artículo original no se implementó en *Minecraft*.

2.3.2 Aprendizaje por Refuerzo

En *Minecraft*, la complejidad y ambigüedad de las recompensas, junto con los largos horizontes temporales, incrementan tanto el coste como la dificultad del entrenamiento. Dentro de estas aproximaciones, los enfoques que alcanzan un buen rendimiento dependen de recursos difíciles de replicar en *hardware* común: VPT [10] se apoya en un preentrenamiento masivo sobre decenas de miles de horas de vídeo, y un *world model* como *DreamerV3* [16] requiere entrenamientos muy prolongados para resolver tareas de horizonte largo. Como respuesta a este coste, la competición *MineRL* [5] surgió para incentivar la eficiencia muestral, y trabajos como el de Scheller et al. [17] mostraron que incorporar demostraciones humanas reduce el número de interacciones necesarias, aunque sin eliminar por completo la dependencia de cómputo.

Marco Teórico

EN este capítulo se presenta una breve explicación de las técnicas implementadas en este trabajo. Se describen los conceptos de planificación simbólica (HTN), aprendizaje por imitación (IL) y aprendizaje por refuerzo (RL), así como las familias de algoritmos en las que se basan.

3.1 Planificación Simbólica

La planificación simbólica se caracteriza por ser un enfoque interpretable debido a su funcionamiento basado en reglas, las cuales definen los estados, acciones y objetivos sobre los que opera el agente, determinando el mundo sobre el que actúa y el plan que debe seguir para cumplir su objetivo. Dentro de la planificación simbólica existen diferentes paradigmas:

- *Stanford Research Institute Problem Solver (STRIPS)* [18]: utiliza un *solver* que, dado un mundo (estado), un conjunto de operadores aplicables (acciones) que transforman el mundo y un objetivo a alcanzar, encuentra una secuencia de acciones que transforma el estado inicial en uno que cumple el objetivo.
- *Boolean Satisfiability Problem (Problema de Satisfacibilidad Booleana) (SAT)* [19]: utiliza técnicas de satisfacibilidad para encontrar planes que cumplan con las restricciones del dominio. Al igual que STRIPS, utiliza un *solver*.
- HTN [8, 20]: con una base similar a STRIPS, pero de forma más jerárquica. En lugar de buscar la secuencia de acciones únicamente a partir del objetivo, las tareas se descomponen recursivamente en subtareas y acciones, formando un árbol de descomposición.

El paradigma seleccionado para este trabajo es el de las HTN, por su capacidad para modelar comportamientos complejos de forma jerárquica, similar a la forma en la que se progresa en *Minecraft*. En lugar de partir únicamente de un objetivo y dejar que el *solver* busque la

secuencia de acciones, una HTN descompone el objetivo en tareas, para las cuales el programador indica cómo se resuelven o cómo se dividen en sub tareas más sencillas. La planificación se convierte entonces, en el recorrido de un árbol de tareas y sub tareas. El comportamiento del agente se basa en tareas, que pueden ser de tres tipos:

- **Primitiva:** se resuelve ejecutando una única acción directa sobre el mundo, siempre que se cumplan sus precondiciones.
- **Compuesta:** no se ejecuta directamente, sino que se descompone mediante uno o varios **métodos**. Cada método es una forma de resolver la tarea usando una secuencia de sub tareas (primitivas u otras compuestas). El uso de varios métodos para la misma tarea permite al **planificador** elegir, según el estado, cómo resolverla.
- **Objetivo:** describe un estado deseado del mundo que el **planificador** debe alcanzar, seleccionando, descomponiendo y ejecutando las tareas (compuestas y primitivas) disponibles.

Como ejemplo, consideremos la tarea de obtener un pico de piedra en *Minecraft*. Se trata de una tarea compuesta que se descompone en una cadena de sub tareas: *conseguir madera*, *fabricar un pico de madera*, *minar piedra* y *fabricar el pico de piedra*. Cada una de ellas es a su vez compuesta y se sigue descomponiendo hasta llegar a primitivas como *minar un bloque* o *fabricar un objeto* en la mesa de trabajo. El estado de tener un pico de piedra en el inventario actúa como indicativo de éxito.

Una de las principales ventajas de las HTN es su capacidad de replanificación. Si una tarea falla durante la ejecución, el **planificador** puede identificar la causa y reaccionar, ya sea modificando el mundo o aplicando un método alternativo para la tarea. En el ejemplo anterior, si el agente intenta minar piedra pero no encuentra ningún bloque accesible, puede explorar el entorno para acercarse a una zona con piedra; y si descubre que ha roto el pico de madera, puede rehacer la sub tarea de fabricarlo antes de continuar.

3.2 Aprendizaje por Imitación

El aprendizaje por imitación (IL) busca obtener una **política** a partir de un conjunto de demostraciones de un experto. Una **política** π_θ , parametrizada por θ , puede ser *estocástica*, asignando a cada estado $s \in \mathcal{S}$ una distribución de probabilidad sobre las acciones ($\mathcal{A}$):

$$\pi_\theta : \mathcal{S} \rightarrow \Delta(\mathcal{A}) \quad (3.1)$$

En esta expresión, la **política** devuelve la probabilidad de tomar la acción a en el estado s (p. ej. *Soft Actor-Critic* (SAC)).

También puede ser *determinista*, cuando devuelve una única acción $a = \pi_\theta(s)$ (p. ej. [DQN](#)), quedando:

$$\pi_\theta : \mathcal{S} \rightarrow \mathcal{A} \quad (3.2)$$

La ventaja de la imitación frente al refuerzo es la eliminación de la función de recompensa, que es el mecanismo que guía el [RL](#), lo que permite al agente aprender comportamientos en entornos complejos sin necesidad de diseñar recompensas manualmente [21]. Dentro del [IL](#) existen múltiples técnicas; en este trabajo se mencionan dos por su buen encaje al problema planteado:

- **Clonado de comportamiento** (*Behavioral Cloning* (BC)): planteado como un problema de [aprendizaje supervisado](#). Dado un conjunto de pares *OBSERVACIÓN-ACCIÓN* generados por el experto, se entrena un modelo que predice la acción que el experto habría tomado. La principal limitación del [BC](#) es el *distribution shift* [15].
- **Métodos interactivos** ([DAGger](#) [15]): el experto ayuda durante el aprendizaje. El aprendiz ejecuta la [política](#) aprendida, el experto etiqueta los estados visitados y estos se incorporan al conjunto de entrenamiento. Esto reduce el *distribution shift* de las técnicas *offline*.

3.3 Aprendizaje por Refuerzo

El aprendizaje por refuerzo ([RL](#)) busca aprender una [política](#) propia mediante interacción con el entorno. El agente realiza acciones, observa el resultado y recibe una recompensa que indica la calidad de su comportamiento, como se ilustra en la Figura 3.1 (página 9).

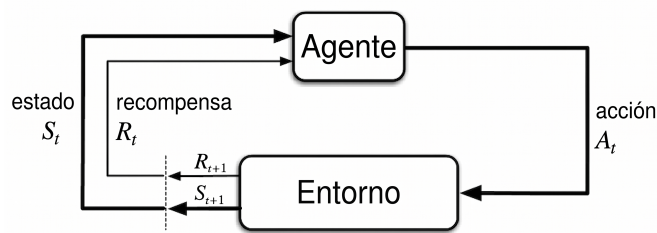


Figura 3.1: Proceso iterativo del aprendizaje por refuerzo. El agente observa el estado del entorno, ejecuta una acción según su [política](#) y recibe una recompensa y un nuevo estado. Adaptada de *Reinforcement Learning: An Introduction* [22].

El problema se plantea habitualmente como un *Markov Decision Process* (Proceso de Decisión de Markov) (MDP) [23], que formaliza la interacción asumiendo que el siguiente estado

y la recompensa dependen únicamente del estado actual y la acción tomada:

$$P(X_{n+1} = j \mid X_n = i, A_n = a) \quad (3.3)$$

donde X_n es el estado en el instante n (con valor i), A_n la acción tomada en ese instante (a) y X_{n+1} el estado resultante (con valor j); es decir, la probabilidad de transitar al estado j depende únicamente del estado y la acción actuales.

El objetivo del agente, por tanto, es maximizar el *retorno esperado*, definido como la suma descontada de las recompensas obtenidas a lo largo de la interacción:

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \right] \quad (3.4)$$

donde r_t es la recompensa recibida en el paso t , $\gamma \in [0, 1]$ es el *factor de descuento* (que resta peso a las recompensas más lejanas) y $\mathbb{E}_\pi[\cdot]$ denota el valor esperado siguiendo la *política* π .

Para evaluar la calidad de las decisiones se define la *función de valor-acción* $Q^\pi(s, a)$, que estima el retorno esperado al ejecutar la acción a en el estado s y seguir después la *política* π :

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a \right] \quad (3.5)$$

Esta función satisface la *ecuación de Bellman*, que divide un problema complejo en subproblemas más simples, expresando el valor de una decisión actual en función de la recompensa inmediata y del valor de los estados futuros. Con ello, se puede expresar de forma recursiva en términos del estado y la acción siguientes (s' y a'):

$$Q^\pi(s, a) = \mathbb{E} \left[r + \gamma \mathbb{E}_{a' \sim \pi} [Q^\pi(s', a')] \right] \quad (3.6)$$

Aunque la ecuación de Bellman propaga teóricamente el valor hacia las decisiones anteriores, en tareas de largo horizonte con recompensas escasas esta propagación se vuelve lenta y la señal de aprendizaje se debilita: el agente apenas recibe recompensa hasta completar una secuencia larga de acciones, lo que dificulta en la práctica atribuir correctamente el valor a cada decisión. Para mitigar esto se utilizan recompensas intermedias (*reward shaping*) que guían la exploración.

Por ejemplo, en la tarea de *woodcutting*, si solo se premiara talar árboles, el agente no aprendería cómo orientarse, moverse o interactuar con el entorno. Por ello se añaden recompensas intermedias por aproximarse al árbol, apuntar o golpearlo.

En este trabajo se analizan tres familias de *DRL*, diferenciadas de los enfoques clásicos por el uso de *aprendizaje automático* para aprender la *política*:

- **Basados en valor:** aprenden una función $Q(s, a)$ y seleccionan la acción de mayor valor. El ejemplo más representativo son las **DQN** [3], que usan redes neuronales y un *replay buffer*.
- **Gradiente de política:** optimizan directamente la **política** mediante ascenso de gradiente. El referente es *Proximal Policy Optimization* (PPO) [24].
- **Actor-crítico:** combinan ambos enfoques, con un actor que produce la **política** y un crítico que evalúa su calidad. El referente es **SAC** [25].

Se añade una taxonomía adicional [22], ilustrada en la Figura 3.2 (página 11):

- **On-policy:** el agente aprende de la **política** que ejecuta.
- **Off-policy:** el agente aprende de experiencias generadas por **políticas** anteriores mediante un *replay buffer*.

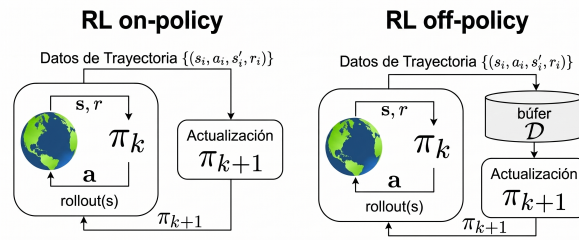


Figura 3.2: Diferencia entre enfoques **on-policy** y **off-policy**. Adaptada de *Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems* [26].

Metodología y planificación

EN este capítulo se describe la metodología seguida durante la realización del proyecto y su planificación. Además, se detallan los conjuntos de datos empleados en el entrenamiento y la evaluación de los modelos.

4.1 Enfoque Metodológico

Al tratarse de un trabajo de investigación con un fuerte componente experimental, se adoptó un enfoque iterativo e incremental. Esta metodología permitió construir el sistema de forma gradual, validando cada técnica antes de incorporar la siguiente y añadiendo valor en cada iteración. A lo largo del proceso se siguieron los principios de la metodología ágil *Scrum* [27], basada en ciclos de trabajo cortos y repetitivos (*sprints*) de una duración media de una semana.

Al final de cada iteración se realizaba una evaluación del progreso, identificando los problemas surgidos y proponiendo soluciones para continuar con el desarrollo. Esta práctica resultó especialmente útil en un proyecto con un alto grado de incertidumbre experimental, en el que el resultado de una técnica condicionaba el diseño de la siguiente. Cada *sprint* concluía con una reunión con los directores del proyecto, en la que se revisaban los avances y se establecían los siguientes pasos.

4.2 Fases del Proyecto

El proyecto se inspira fuertemente en el estado del arte (Capítulo 2), con el que comparte algunos conceptos de diseño (conjunto de acciones, estados, recompensas...), pero del que difiere en su implementación al probar familias de técnicas muy variadas entre sí, incluyendo algoritmos no implementados directamente en *Minecraft*, como las HTN. A continuación se presentan tanto la planificación inicial como las fases finales en las que se dividió el trabajo.

4.2.1 Fases *a priori*

Antes de comenzar el desarrollo se definieron las siguientes cuatro fases:

1. **Análisis y preparación inicial:** estudio del estado del arte sobre agentes en *Minecraft* y familiarización con las herramientas (entorno de simulación, comunicación con el agente y *frameworks*).
2. **Planificación simbólica:** implementación de la HTN como primera técnica y *baseline* interpretable para la comparativa.
3. **Aprendizaje por refuerzo:** implementación de un agente de refuerzo capaz de resolver la tarea únicamente a partir de la observación del entorno.
4. **Ejecución y evaluación final:** ejecución de las técnicas, configuración necesaria y análisis comparativo de los resultados.

4.2.2 Fases *a posteriori*

En el desarrollo real, el trabajo se dividió en siete iteraciones. Debido a que la aproximación incremental, el diseño y la realización de cada fase condicionaban la siguiente. Por ejemplo, la fase de HTN se usó para generar un conjunto de demostraciones que sirvió como base para el IL, el cual se añadió con posterioridad a la planificación inicial.

1. **Análisis y preparación inicial:** revisión del trabajo previo, junto con la familiarización con el entorno de simulación y el diseño de la arquitectura general del sistema, descrito en el Capítulo 5.
2. **Entorno de simulación:** implementación de un entorno *Gymnasium* comunicado mediante HTTP con un agente *Mineflayer* [28] sobre un servidor real de *Minecraft*, con reinicio determinista.
3. **Planificación simbólica y generación del *dataset*:** implementación del planificador HTN y de la descomposición de la tarea de *woodcutting*, empleando sus ejecuciones como conjunto de demostraciones para la fase de IL.
4. **Aprendizaje por imitación:** diseño iterativo y entrenamiento final de una política con arquitectura de doble cabeza sobre el *dataset* generado por el HTN.
5. **Aprendizaje por refuerzo:** diseño iterativo y entrenamiento de una política de refuerzo, optimizando la arquitectura y los hiperparámetros.
6. **Evaluación:** introducción de métricas, definición de una condición de éxito explícita y ajuste de la función de recompensa.

7. **Resultados y evaluación final:** evaluación de los resultados de todas las implementaciones, analizando sus fortalezas y debilidades.

Durante el desarrollo se exploraron además otras dos aproximaciones que finalmente se descartaron del alcance de la comparativa. La primera fue el uso de agentes basados en LLMs mediante el *framework* *Minecraft* [14], descartada por el elevado coste computacional de ejecutar LLMs pesados en local. La segunda fue un detector visual de árboles basado en YOLO [29], descartado por desplazar el objetivo hacia un problema de detección de objetos en lugar del aprendizaje de la tarea en sí.

4.3 Recursos y limitaciones

Para el desarrollo del proyecto se emplearon dos equipos: un ordenador de sobremesa, encargado de ejecutar los entrenamientos, alojar el servidor de *Minecraft*, ejecutar el agente y el desarrollo general; y un portátil MacBook Air M3, empleado como equipo secundario. Sus características se resumen en la Tabla 4.1 (página 14).

Característica	Sobremesa	MacBook Air M3
CPU	AMD Ryzen 9 5900X (12 C/24 T)	Apple M3 (8 núcleos)
RAM	32 GB	16 GB
GPU	NVIDIA RTX 3060 12 GB	Integrada (10 núcleos)
Uso	Entrenamiento y desarrollo	Equipo complementario

Tabla 4.1: Equipos empleados en el desarrollo del proyecto.

La principal limitación de recursos fue el tiempo de entrenamiento. Si bien el sistema no generaba una gran carga computacional, la ejecución de las acciones suponía una limitación temporal. El entorno de simulación era el cuello de botella a la hora de procesar las acciones. Si el agente realiza la acción de talar y esta requiere 2 a 3 segundos para completarse, en un episodio de 100 pasos el tiempo de ejecución asciende a 200 a 300 segundos. Considerando la ejecución completa, con 300 episodios se alcanzan entre 16 y 25 horas de entrenamiento, lo que limita el número de experimentos que se pueden realizar.

La única paralelización posible sería ejecutar varios agentes a la vez o usar varios equipos simultáneamente, pero no se hizo debido a la limitación de recursos computacionales disponibles y a la dificultad de coordinar el desarrollo en varios equipos a la vez.

4.4 Conjuntos de Datos

A diferencia de un problema de clasificación supervisada con conjuntos de datos estáticos, en este trabajo los datos provienen de la interacción de agentes con el entorno. Se consideraron dos conjuntos de demostraciones, seleccionando el primero como conjunto final.

Dataset HTN: este conjunto de datos de elaboración propia se compone de las trayectorias producidas por el agente simbólico al resolver la tarea de *woodcutting*. Estas trayectorias se registran como pares *OBSERVACIÓN-ACCIÓN* (definidos en el Capítulo 5) y conforman el conjunto de demostraciones de un experto determinista e interpretable. Este *dataset* permite la comparación entre el comportamiento simbólico y las técnicas de aprendizaje profundo. Se realiza una pequeña limpieza descartando las muestras de acciones poco representativas, quedando un total de 118 episodios y 4 094 *steps* (4 069 tras la limpieza), como se detalla en la Tabla 4.2 (página 15).

Dataset MineRL Treechop-v0: este conjunto de demostraciones humanas [5] se compone de 210 episodios (todos con éxito) y aproximadamente 453 000 *steps*, con una mediana de 64 troncos recogidos por episodio. Su análisis exploratorio reveló que el movimiento de cámara humano (mediana de $0,75^\circ/0,90^\circ$ por *step*) es aproximadamente diez veces más fino que el paso de cámara de la discretización empleada por los agentes discretos, lo que impide trasladar directamente las demostraciones humanas a su espacio de acción. Además, el *dataset* humano presenta limitaciones al ser utilizado fuera del entorno de desarrollo de *MineRL*. Entre estas limitaciones se encuentran: la estructura de los datos, el formato de las acciones y el formato de las observaciones, lo que dificulta su uso directo para el entrenamiento de agentes en nuestro entorno de simulación. Por último, *MineRL* solo proporciona información visual, por lo que no permite obtener el vector de estado ni las señales de percepción (*tree_visible*, *tree_distance*) de las que dependen nuestros agentes.

Propiedad	htn	MineRL Treechop-v0
Origen	Planificador simbólico (propio)	Demostraciones humanas
Episodios	118	210
<i>Steps</i> (bruto)	4 094	~453 000
<i>Steps</i> (limpio)	4 069	~453 000

Tabla 4.2: Comparación de los conjuntos de datos analizados en el trabajo.

Cabe mencionar además que el *framework* de *MineRL* se intentó utilizar, pero actualmente ya no recibe mantenimiento y no es compatible con las versiones actuales de *Gymnasium* [30].

Por otra parte, este tipo de *datasets* están pensados para ser utilizados dentro de su propio *framework*, lo que dificulta su uso en entornos de simulación externos como el empleado en este proyecto.

Arquitectura del Sistema

EN este capítulo se describe la arquitectura general del sistema. Aunque cada técnica utiliza componentes específicos para su funcionamiento, todas se implementan sobre una misma infraestructura.

Esta decisión permite reutilizar los mecanismos de interacción con el entorno y mantener condiciones experimentales idénticas entre las tres aproximaciones comparadas. Las particularidades de cada una se describen en sus respectivos Capítulos 6, 7 y 8.

5.1 Arquitectura General

El sistema se organiza en tres capas que se comunican entre sí, como se resume en la Figura 5.1 (página 18): (i) **Capa de simulación**, un servidor real de *Minecraft*; (ii) **Capa de agentes**, control del agente y obtención de los estados del mundo; y (iii) **Capa de decisión**, comunicación con los modelos de aprendizaje.

5.1.1 Capa de Simulación

Como se menciona en el Capítulo 2, la mayoría de los trabajos relacionados con *Minecraft* emplean entornos de simulación propios, basados en versiones modificadas del juego como *MineRL* [5]. Sin embargo, muchas de estas librerías están actualmente abandonadas y presentan problemas de compilación [30], por lo que se opta por un servidor real de *Minecraft* lanzado en modo *headless* basado en *Paper*¹ [31].

El mundo utilizado tiene la misma *semilla*² en todas las iteraciones para evitar mundos donde el agente aparezca en *biomas* sin árboles y garantizar el mismo entorno de prueba para todas las técnicas.

¹ Una implementación de alto rendimiento del servidor de *Minecraft*.

² Código numérico que funciona como plantilla para generar el mapa. Determina dónde aparecen los *biomas*, las estructuras, los minerales y el terreno, de modo que la misma *semilla* produce siempre el mismo mundo.

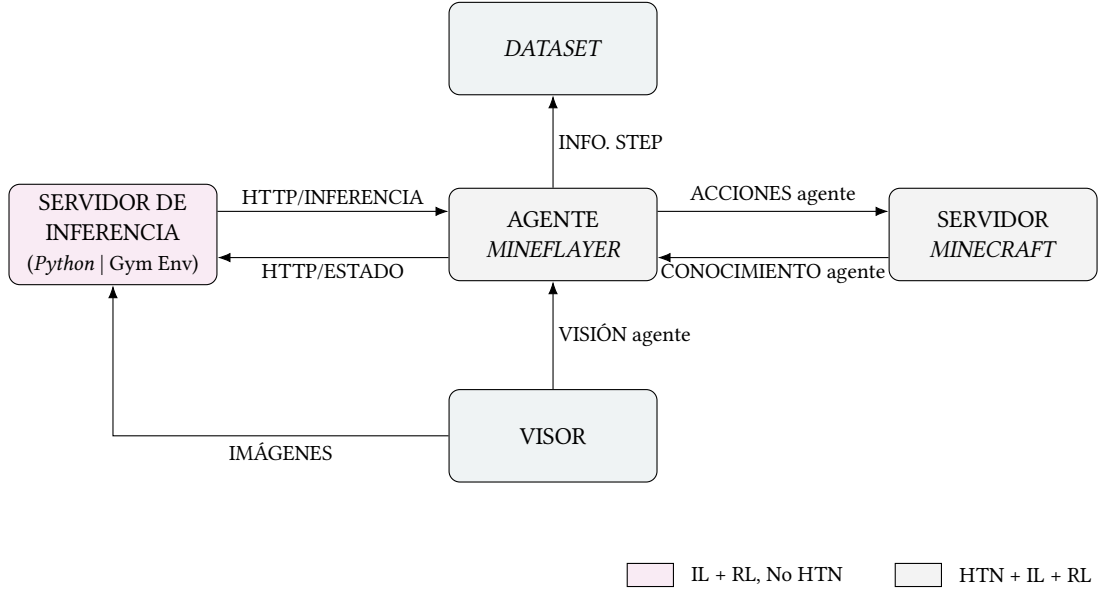


Figura 5.1: Arquitectura general del sistema. La capa de decisión (*Python*, servidor de inferencia para *IL* y entorno *Gymnasium* para *RL*) se comunica por *HTTP* con el agente. La *HTN* reside por completo en el agente, sin capa de decisión.

Si bien usar el mismo mundo podría limitar la generalización, se introduce un mecanismo de reinicio con puntos de aparición aleatorios para aportar cierta diversidad de escenarios.

Esta diversidad de escenarios y la reproducibilidad de los episodios se basan en un reinicio determinista. El reinicio de episodio elimina al agente del mundo y lo teletransporta a una posición de aparición fija (la primera al entrar al mundo), de modo que cada episodio comienza con la misma posición dentro del mismo mundo, condición necesaria para que el agente pueda explorar el entorno. El reinicio de mundo detiene el servidor, borra el mundo y arranca uno nuevo; se puede emplear la misma *semilla* (para repetir el mismo escenario) o una aleatoria (para generar mundos aleatorios y evaluar la generalización en distintos entornos).

El agente se conecta al servidor como un jugador mediante *Mineflayer* [28], una *API* en *Node.js* [32] que actúa como interfaz entre el código del agente y el mundo de *Minecraft*.

A través de ella se leen las percepciones (posición, orientación, inventario y bloques cercanos) y se envían las acciones (movimiento, rotación de cámara, minado e interacción con bloques). La percepción visual se obtiene de *prismarine-viewer*, un plugin de *Mineflayer* que renderiza la vista en primera persona del agente en un puerto local (véase el Apéndice B); *Puppeteer* [33] captura esas imágenes y las envía a los modelos o las guarda en disco para el *dataset* de entrenamiento.

5.1.2 Capa de agentes

Toda la lógica de control gira en torno a una clase base, *BaseAgent* (Figura 5.2, página 19), que contiene elementos comunes de los agentes: conexión, punto de aparición en el mundo, inicialización del visor en primera persona, gestión de errores y desconexión limpia liberando puertos y recursos. De ella heredan las tres implementaciones de las técnicas:

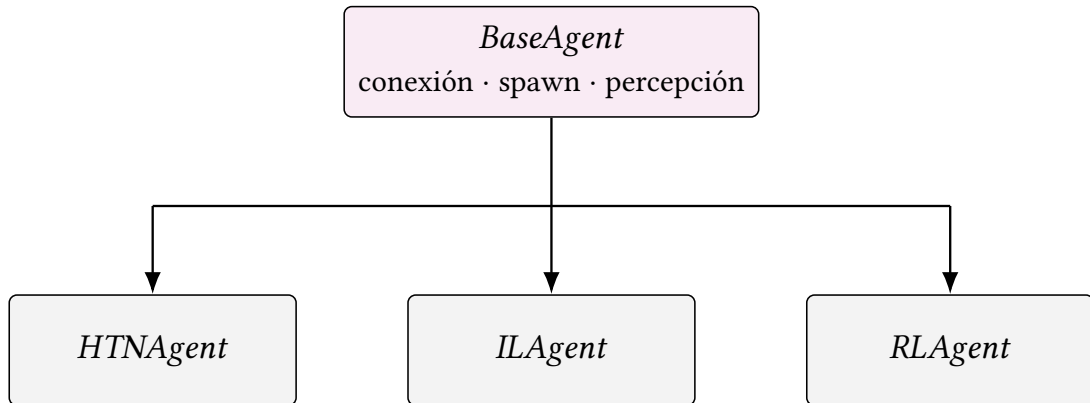


Figura 5.2: Jerarquía de clases de la capa de agentes. La clase base *BaseAgent* concentra la conexión, el punto de aparición y la percepción comunes; de ella heredan *HTNAgent*, *ILAgent* y *RLAgent*, cada una con su propia lógica de decisión.

- *HTNAgent*: ejecuta la [HTN](#) directamente, sin intervención externa. Su lógica reside por completo en las capas de *Agente* y *Simulación*.
- *ILAgent*: en cada paso captura una imagen del visor, la envía al servidor de inferencia, recibe la acción predicha (acción discreta y delta de cámara) y la ejecuta.
- *RLAgent*: actúa como puente entre el entorno [Gymnasium](#) y el agente, exponiendo un servidor [HTTP](#) que recibe acciones, las ejecuta y devuelve la observación y los eventos.

Las tres técnicas comparten la conexión y la percepción, pero difieren en la forma en que deciden la siguiente acción. Los cambios particulares de las implementaciones se detallan en sus respectivos Capítulos 6, 7 y 8.

5.1.3 Capa de decisión

La comunicación entre la capa de decisión (*Python*) y la de agentes (*Node.js*) se realiza por [HTTP](#) con formatos [JSON](#). Se emplean dos canales según la técnica:

- **Imitación**: el *ILAgent* actúa como cliente de un servidor de inferencia *Python* con un [endpoint](#) `/predict`, al que envía la imagen del visor y el vector de estado, y del que recibe la acción predicha por el modelo.

- **Refuerzo:** el *RLAgent* levanta un servidor [HTTP](#) con tres *endpoints*: */step*, que recibe una acción, la ejecuta y devuelve la nueva observación junto con los eventos (bloques rotos, ataque a un árbol, muerte del agente...); */reset*, que reinicia el episodio; y */world_reset*, que regenera el mundo.

5.2 Entorno de simulación: Minecraft

Como ya se describió en el Capítulo 1, *Minecraft* es un videojuego 3D de mundo abierto en primera persona, basado en bloques y de generación procedimental. La Figura 5.3a (página 20) muestra la vista en primera persona de un jugador humano, y la Figura 5.3b (página 20) muestra el sistema de coordenadas que define su orientación y posición en el mundo.

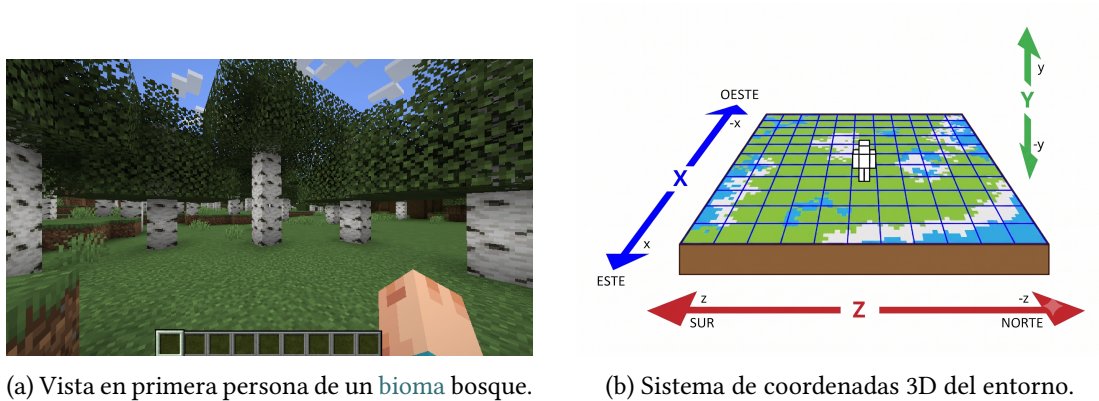


Figura 5.3: Entorno espacial de *Minecraft*: vista en primera persona con el [Interfaz de Información en Pantalla, Heads-Up Display \(HUD\)](#) activado (izquierda) y representación del sistema de coordenadas (derecha).

A diferencia de juegos 2D como *Atari Breakout* [3], el entorno 3D introduce una nueva dimensión de complejidad, dificultando así las tareas de navegación, orientación y percepción visual.

5.3 Formulación de la tarea

La tarea común a todas las técnicas es el [woodcutting](#). El agente debe localizar un árbol, navegar hacia él, talarlo y recoger la madera.

Esta acción es la que realizaría cualquier jugador humano al comenzar a jugar a *Minecraft*, y es el primer paso de la jerarquía de progresión del juego. Aunque parezca una tarea sencilla desde la perspectiva humana, presenta una gran dificultad para los agentes autónomos: percepción visual, navegación, orientación de la cámara, reconocimiento de objetos, recompensa

compleja...En las siguientes secciones se detallan los elementos comunes de la formulación de la tarea, adaptados posteriormente a cada técnica.

5.3.1 Espacio de observaciones

El vector de estado se compone de ocho elementos. Las variables continuas (*yaw*, *pitch*, *dx*, *dz*, *tree_distance* y *log_count*) se escalan al rango $[-1, 1]$ mediante límites fijos conocidos del entorno, propios de cada variable: los ángulos *yaw* y *pitch* a $[-\pi, \pi]$ y $[-\pi/2, \pi/2]$ respectivamente; los desplazamientos del movimiento *dx* y *dz* se recortan a $[-1, 1]$ ya que el agente avanza como máximo un bloque por paso; *tree_distance* a $[0, 40]$ bloques (en un *bioma* de bosque siempre hay un árbol cerca); y *log_count* a $[0, 64]$ (un *stack* de inventario). Las variables binarias (*tree_visible* e *is_looking_at_log*) se transforman a $\{-1, 1\}$ para mantener una escala homogénea en las entradas de la red [34]. La Tabla 5.1 (página 21) resume los elementos del vector de estado.

Componente	Rango	Descripción
<i>yaw</i>	$[-\pi, \pi]$	Orientación horizontal de la cámara (rad)
<i>pitch</i>	$[-\pi/2, \pi/2]$	Orientación vertical de la cámara (rad)
<i>dx</i>	$[-1, 1]$	Desplazamiento en X respecto al paso anterior
<i>dz</i>	$[-1, 1]$	Desplazamiento en Z respecto al paso anterior
<i>tree_visible</i>	$\{-1, 1\}$	Indicador de árbol visible en el cono de visión
<i>tree_distance</i>	$[0, 40]$	Distancia al árbol visible más cercano
<i>log_count</i>	$[0, 64]$	Troncos acumulados en el inventario
<i>is_looking_at_log</i>	$\{-1, 1\}$	Indicador de cursor apuntando a un tronco

Tabla 5.1: Elementos del vector de estado.

La posición absoluta (x, y, z) se excluye del vector de estado de los agentes para favorecer la generalización. Con coordenadas absolutas, el agente podría tender a memorizar posiciones concretas del mundo en lugar de aprender a talar. En su lugar se conservan los desplazamientos entre pasos (*dx*, *dz*), que aportan información útil. Del mismo modo, solo se conserva el desplazamiento horizontal (*dx*, *dz*) y se omite el vertical (*dy*). En el *bioma* utilizado el movimiento vertical es prácticamente nulo, por lo que aportaría un componente casi constante y sin información, al ser la mayoría de los valores 0. Además, la entrada visual codifica de manera implícita la altura del agente respecto a su entorno, aportando un contexto espacial mucho más rico y útil que la inclusión de una coordenada vertical en el vector de estado. Cuando se habilita la entrada visual, además del vector de estado, la red recibe una imagen RGB de 128×128 píxeles capturada del visor en primera persona. En ambos casos se admite el *frame-stacking*, repetir las últimas *k* observaciones, para dar información temporal a la red.

5.3.2 Espacio de acciones

El espacio de acciones utilizado consta de siete acciones recogidas en la Tabla 5.2 (página 22). La cámara se discretiza en giros fijos: 0,15 rad ($\approx 8,6^\circ$) en horizontal y 0,10 rad ($\approx 5,7^\circ$) en vertical. Se utiliza esta granularidad para conseguir un cambio de estado real, sin que el agente gire la cámara manteniéndose en el mismo estado.

Acción	Efecto
<i>attack</i>	Minar el bloque apuntado por el cursor
<i>move_forward_sprint</i>	Avanzar corriendo
<i>move_forward_jump</i>	Avanzar corriendo con salto
<i>camera_right</i>	Girar la cámara a la derecha
<i>camera_left</i>	Girar la cámara a la izquierda
<i>camera_up</i>	Subir la cámara
<i>camera_down</i>	Bajar la cámara

Tabla 5.2: Espacio de acciones discreto.

5.3.3 Condiciones de terminación y éxito

Un episodio puede finalizar de tres formas. La primera es el **éxito**: recoger al menos el número de troncos objetivo k y, además, haber roto al menos un tronco durante el episodio. Esta segunda condición previene un comportamiento espurio donde el agente recoge troncos de episodios anteriores sin talar nuevos árboles.

La segunda forma es la **terminación anticipada**, que se dispara si el agente muere o si la recompensa acumulada cae por debajo de un umbral, cortando episodios sin progreso. Al ser un final real de la tarea, se deja el valor del estado siguiente a cero.

La última forma es el **truncamiento**, al alcanzar el número máximo de pasos por episodio. A diferencia de la terminación, el valor del estado siguiente no es cero, lo que indica al agente que el episodio aún podría continuar.

5.3.4 Percepción del entorno

El componente principal de la percepción es la detección de árboles. Si bien el agente podría obtener un conocimiento omnisciente del mundo a través de la [API](#) de *Mineflayer*

(conociendo la posición exacta de cualquier bloque, enterrado o tras un muro), se limita la información que recibe para simular lo máximo posible la percepción de un jugador humano.

Para ello, la búsqueda de troncos se restringe al *Field of View* (Campo de Visión) (FOV) del agente, evitando información sobre objetos que no podría ver. Además, para percibir un objeto el agente debe tener una línea de visión directa hacia él.

La detección comienza con la función *findBlocks* de *Mineflayer*, que devuelve las posiciones de los bloques de un tipo dado dentro de un radio fijo (por defecto, 32 bloques) alrededor del agente. Los bloques devueltos por la función vienen ordenados por cercanía al agente. Sobre esos candidatos se aplica un filtro en tres etapas, devolviendo el primer bloque que las supera todas:

1. **Cara expuesta:** el bloque debe tener al menos una cara al aire. Esto asegura que el bloque es visible y accesible directamente.
2. **Cono de visión:** el bloque debe estar dentro del campo de visión del agente, cono de 90° en horizontal y 60° en vertical respecto a la orientación (*yaw*, *pitch*) del agente.
3. **Línea de visión:** se traza un *raycast* (línea proyectada desde un punto fijo en una dirección concreta) desde los ojos del agente hasta el bloque, el cual debe alcanzarse sin chocar previamente con otro bloque sólido (elementos como el agua no bloquean la visión).

De este modo, el agente solo conoce los árboles que un jugador podría realmente ver en ese instante.

5.3.5 Reproducibilidad

La reproducibilidad del sistema se basa en el uso del mismo mundo y *semilla* en todas las iteraciones y en los reinicios deterministas a una posición de aparición fija. Así, todas las técnicas se entrenan y evalúan bajo idénticas condiciones, variando únicamente en la posición inicial al generar el mundo, lo que garantiza que las comparaciones entre las familias de algoritmos del Capítulo 9 sean justas.

Iteración I - Agente HTN

EN esta primera iteración, se aborda la planificación simbólica usando un planificador jerárquico de tareas (HTN), que sirve de base para la comparación con el resto de algoritmos gracias a su carácter determinista y explicable.

Como resultado de esta iteración se obtienen dos elementos: un agente que actúa como *baseline* frente a las técnicas de aprendizaje, y un *dataset* propio de demostraciones generado a partir de sus ejecuciones para entrenar en el Capítulo 7.

6.1 Diseño

La tarea objetivo *woodcutting* consiste en la recolección de k troncos de un tipo específico de madera. Esta tarea objetivo se descompone en una tarea compuesta que se resuelve aplicando repetidamente una secuencia de primitivas (Figura 6.1, página 24).



Figura 6.1: Árbol de descomposición HTN de la tarea de *woodcutting*. La tarea compuesta se descompone en una primitiva de selección de madera y en una tarea compuesta de tala que se repite hasta conseguir los k troncos. Las tareas **compuestas** se muestran en **azul**, las **primitivas** en **verde** y la **recuperación** en **rojo**.

Cada tarea se define mediante: una **pre-condición**, estado del mundo antes de la ejecución; una **post-condición**, estado del mundo después de la ejecución; y un **método**, forma de resolver la tarea usando una secuencia de subtareas (primitivas u otras compuestas). Estas restricciones son las que otorgan al agente su carácter **determinista** (ante el mismo estado toma siempre la misma decisión) e **interpretable** (cada acción corresponde a una regla).

La condición de éxito de la tarea de *woodcutting* es tener k troncos en el inventario. Al ser una tarea objetivo, no se ejecuta de forma directa, sino que se descompone en la tarea compuesta de talar un árbol, que se repite hasta alcanzar los k troncos. Talar un árbol implica, a su vez, varios pasos: localizar un tronco dentro del FOV (y, si no hay ninguno, explorar el mundo hasta que aparezca uno), desplazarse hasta él y, finalmente, talar el tronco completo. Cada uno de estos pasos se descompone de nuevo hasta llegar a primitivas como mover la cámara o minar un bloque.

Al ser una de las primeras tareas a realizar en *Minecraft*, la tarea *woodcutting* no presenta realmente pre-condiciones, ya que el agente puede empezar a talar en cualquier momento. Sin embargo, en tareas posteriores como la obtención de un pico de hierro es necesario que el agente tenga las herramientas adecuadas (véase el Apéndice C).

6.2 Tareas Primitivas

Las tareas primitivas que componen la tarea de *woodcutting* se recogen en la Tabla 6.1 (página 25). Cada primitiva se traduce en una acción básica realizable en *Minecraft* (moverse, minar, mover la cámara...) y se apoya en la API de *Mineflayer* descrita en el Capítulo 5.

Primitiva	Función
<i>obtainWoodType</i>	Determinar el tipo de madera
<i>findNearestVisibleBlock</i>	Localizar el tronco visible más cercano (FOV)
<i>moveToBlock</i>	Navegar al bloque mediante un <i>pathfinder</i>
<i>collectBlock</i>	Obtener un bloque y recogerlo
<i>mineTreeTrunk</i>	Talar una secuencia de bloques
<i>exploreRandom</i>	Explorar si no hay árbol visible (recuperación)

Tabla 6.1: Primitivas del agente simbólico para la tarea de *woodcutting*.

Selección del tipo de madera: utilizando la API de *Mineflayer* se identifica el *bioma* y, mediante un mapeo predefinido, se determina el tipo de madera objetivo a talar (por ejemplo, *forest* → *oak_log*); si el bioma no está mapeado, se recurre a una búsqueda visual de los tipos

de tronco más comunes en el entorno. Si no hay madera visible al alcance, la tarea falla en lugar de consultar la posición omnisciente de los bloques, en coherencia con la restricción de percepción del Capítulo 5.

Búsqueda de Bloques en el Cono de Visión: la localización de troncos reutiliza el mismo mecanismo de percepción definido en el Capítulo 5. La búsqueda se restringe al FOV del agente y exige línea de visión directa, de modo que el agente solo considera los árboles que un jugador humano vería. La primitiva de talar usa esta búsqueda para evitar información del mundo fuera del FOV.

Tala del Tronco: una vez localizado el bloque, la tala no vuelve a invocar la búsqueda por FOV para los bloques superiores. En su lugar, *mineTreeTrunk* recorre verticalmente la posición ya conocida: mina el bloque en $(x, y+1, z)$, comprueba si el bloque en $(x, y+n, z)$ es del mismo tipo de tronco, y así sucesivamente hasta que el tipo de bloque cambia. Esta estrategia permite talar un árbol completo sin necesidad de reposicionarse ni relanzar el *pathfinder* para cada bloque.

Exploración de Recuperación: cuando la búsqueda en el FOV no encuentra ningún árbol, el agente no puede determinar si existe madera en el mundo o si simplemente no es visible desde su posición actual. En este caso se activa *exploreRandom*, que elige una dirección aleatoria y pide al *pathfinder* avanzar hasta 30 bloques en esa dirección, permaneciendo en la superficie para evitar posiciones subterráneas donde no se encuentra madera. Durante la exploración, se comprueba cada 500 ms si ha aparecido algún tronco en el FOV; si es así, la exploración se interrumpe y se retoma la tala.

Progresión y Recuperación de Fallos: la ejecución de cada tarea contiene una lógica de recuperación (Figura 6.2, página 27). Antes de actuar se comprueba la post-condición; mientras no se cumpla, se ejecuta el método de la tarea hasta un máximo de N intentos. Si un intento lanza un error (por ejemplo, el objetivo ha desaparecido), se dispara un mecanismo de recuperación: una acción de recuperación específica de la tarea o, si no existe, una exploración aleatoria del entorno para reposicionar al agente antes de reintentar. Si el agente se queda sin intentos, la tarea se aborta y se propaga el fallo a la tarea padre, o bien, se detiene la ejecución si es la tarea raíz, marcando la ejecución como fallida.

Se añaden dos prevenciones extra debido a las limitaciones de la simulación. La primera consiste en una comprobación periódica de la posición del agente mientras el *pathfinder* está activo; si no avanza lo suficiente durante varias muestras consecutivas (por ejemplo, si el agente se queda atascado), se reinicia la navegación. La segunda es un *timeout* global por episodio que limita la duración máxima de una ejecución para evitar bucles infinitos.

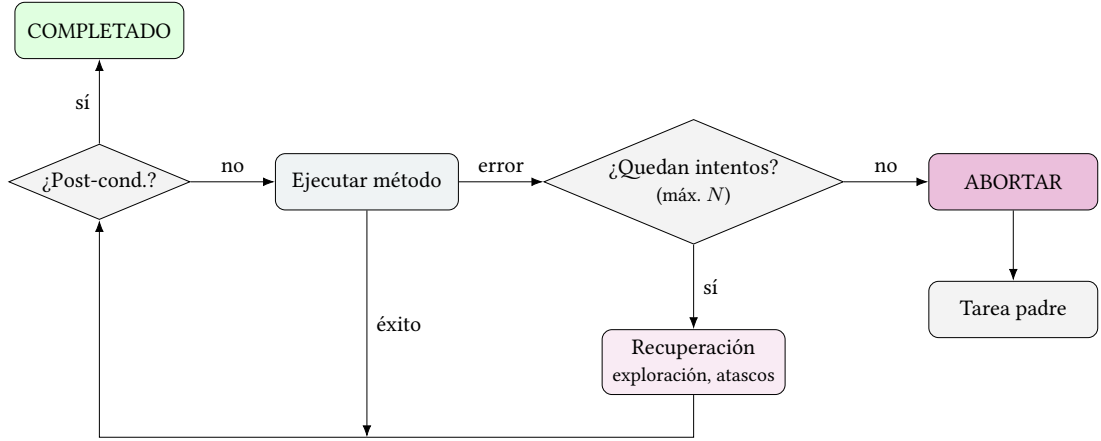


Figura 6.2: Bucle de ejecución de una tarea (*runSmartTask*) del HTN. Antes de cada intento se comprueba la post-condición; mientras no se cumpla, se ejecuta el método de la tarea. Si un intento falla, se dispara un mecanismo de recuperación (una acción específica o una exploración aleatoria) y se reintenta, hasta un máximo de N intentos. Si se agotan, la tarea se aborta y propaga el fallo a la tarea padre.

6.3 Generación del Dataset de Demostraciones

Al comenzar la implementación del agente de imitación (Capítulo 7) surgió la necesidad de un conjunto de demostraciones de un experto. Una de las opciones sería reutilizar un *dataset* público como *MineRL* [5], cuyo subdataset *Treechop-v0* aborda precisamente la misma tarea. Sin embargo, su uso resulta imposible en este trabajo por los motivos mencionados en el Capítulo 4.

Por todo ello, en lugar de adaptar un *dataset* ajeno se genera uno propio ejecutando el agente HTN sobre el mismo entorno en el que se evalúan las técnicas. Al compartir exactamente el mismo entorno, espacio de acciones y espacio de observación que el agente de imitación, las demostraciones no necesitan ninguna conversión y sus etiquetas coinciden con el conjunto de acciones del modelo. Esto garantiza la coherencia entre el experto y el aprendiz y elimina el desajuste de formato y granularidad de *datasets* externos.

Durante la ejecución se almacenan pares *OBSERVACIÓN-ACCIÓN* que conformarán el *dataset*. A intervalos regulares se captura un *frame* que combina la imagen del visor en primera persona (véase el Apéndice B), el vector de estado del agente de la Tabla 5.1 (página 21) y la acción que está ejecutando en ese instante. Cabe destacar que no se incluyen los movimientos de la cámara como acciones discretas, ya que el agente los realiza sobre posiciones absolutas del mundo; es decir, no emplea acciones como *camera_right*, sino que orienta su visión directamente mediante funciones como *lookAt(x,y,z)*. Los *frames* sin acción se descartan (por ejemplo, los capturados durante el tiempo de inicialización, donde el agente aún no interactúa

con el entorno). La estructura final de estos *frames* se detalla en la Tabla 6.2 (página 28).

Campo	Tipo / Rango	Descripción
<i>image</i>	PNG	Vista en primera persona del visor
<i>x, y, z</i>	float (bloques)	Posición del agente
<i>yaw, pitch</i>	float (rad)	Orientación de la cámara
<i>action</i>	categorica	Acción discreta ejecutada en el <i>frame</i>
<i>dyaw, dpitch</i>	float (rad)	Desplazamiento de cámara respecto al <i>frame</i> anterior
<i>tree_visible</i>	{0, 1}	Árbol dentro del FOV
<i>tree_distance</i>	float (bloques)	Distancia al árbol visible más cercano

Tabla 6.2: Estructura de los *frames* del *dataset* de demostraciones generado por el HTN. La posición absoluta se guarda para calcular el desplazamiento entre *frames* posteriormente.

La distribución de acciones del *dataset*, ilustrada en la Figura 6.3 (página 28), está desbalanceada. Esto se debe a que el agente simbólico dispone de información completa del entorno a través de la *API*, lo que le permite actuar de forma muy eficiente. Por ejemplo, conoce la posición exacta de los árboles y, mediante el uso del *pathfinder*, calcula rutas óptimas que rara vez incluyen movimientos laterales o hacia atrás. Como consecuencia, aunque el espacio de acciones contiene acciones como saltar, moverse a la derecha o moverse a la izquierda, el agente solo ejecuta las más eficientes para completar la tarea. Este comportamiento provoca que ciertas acciones aparezcan con muy poca frecuencia en el *dataset*, lo que motiva el descarte de las clases menos representadas.

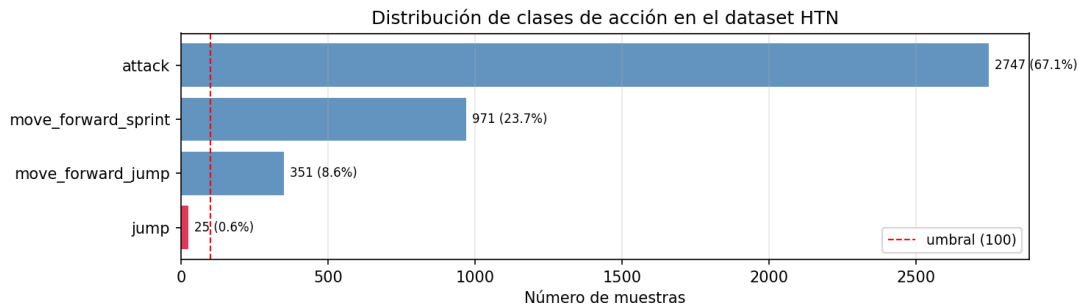


Figura 6.3: Distribución de las clases de acción en el *dataset* de demostraciones generado por el agente simbólico tras 118 ejecuciones.

El registro de la acción discreta y la variación de la cámara es lo que permite entrenar una arquitectura de doble cabeza (clasificación de la acción y regresión del movimiento de cámara correspondiente a la acción) en el Capítulo 7. Tras una limpieza que descarta episodios con errores y acciones poco representativas, el *dataset* resultante consta de los episodios y *steps* detallados en la Tabla 4.2 (página 15).

6.4 Resultados de la Iteración

El agente simbólico se evalúa sobre la misma tarea de entrenamiento, *woodcutting*, ejecutando un conjunto de episodios con reinicio determinista (Capítulo 5). Cada episodio registra: los troncos recogidos, los troncos rotos y la duración; la tasa de éxito se calcula con la métrica común de todas las técnicas (recoger al menos k troncos, con $k = 5$), lo que garantiza una comparación justa en el Capítulo 9.

Sobre un total de 50 episodios, el agente alcanza una tasa de éxito del 100,0 %. La distribución por episodio de troncos, tiempo y pasos se resume en la Tabla 6.3 (página 29).

Técnica	Eps.	Éxito (%)	Troncos			Tiempo (s)		
			μ	$\tilde{x}$	σ	μ	$\tilde{x}$	σ
HTN	50	100,0	6,00	6,00	0,00	26,7	26,4	1,1

Tabla 6.3: Evaluación de la técnica HTN bajo la métrica común recoger $k = 5$ troncos. Troncos y tiempo por episodio: media (μ), mediana ($\tilde{x}$), y desviación típica (σ).

Como *baseline*, el agente simbólico ofrece un comportamiento robusto e interpretable sin necesidad de entrenamiento. La limitación principal es la dependencia del conocimiento del dominio definido manualmente: el rendimiento está condicionado por la calidad de las primitivas y de la navegación, y los fallos pueden provenir sobre todo de situaciones de atasco o de escenarios en los que no hay madera visible al alcance. La tarea empleada para la evaluación es una de las más simples de *Minecraft*, precedida solo por la de *navegación*. Las siguientes tareas a realizar según el árbol de progresión del juego (véase el Apéndice C) presentan un incremento significativo de complejidad, al requerir un mayor número de subtareas y dependencias entre recursos.

Para la realización de las tareas el agente HTN utiliza información que no es común para el jugador (posiciones exactas, visión y movimiento perfectos...); aunque sí obtenible, manteniendo la restricción de omnisciencia. Esto plantea la necesidad de recurrir al *aprendizaje automático*, utilizando la misma información que usaría un jugador real. Si bien la obtención de los datos para el enfoque automático puede ser una tarea ardua, su horizonte puede resultar menos limitado que el del enfoque simbólico.

6.5 Escalabilidad a tareas de horizonte largo

La HTN no se limita a la tala de madera. La misma arquitectura de tareas compuestas, primitivas y recuperación escala a la progresión completa hasta el pico de hierro (véase el Apéndice C), que encadena tres fases: madera, piedra y hierro.

6.5.1 Dependencias entre tareas: precondiciones

En el *woodcutting* la única dependencia real es conocer el tipo de madera del *bioma*. En cambio, la progresión hasta el hierro encadena requisitos estrictos: no se puede minar piedra sin un pico de madera, ni hierro sin un pico de piedra, ni fundir sin un horno.

El planificador gestiona esto mediante precondiciones recursivas: antes de ejecutar una tarea comprueba si se cumple el estado que necesita y, si no, lo resuelve como subtarea. Por ejemplo, minar piedra exige un pico de madera (*ensureEquipped*); si el agente no lo tiene, lo fabrica, lo que a su vez requiere tablones y una mesa de trabajo (*ensureWoodAndPlanks*, *ensureCraftingTable*).

La Figura 6.4 (página 30) ilustra ese descenso recursivo.

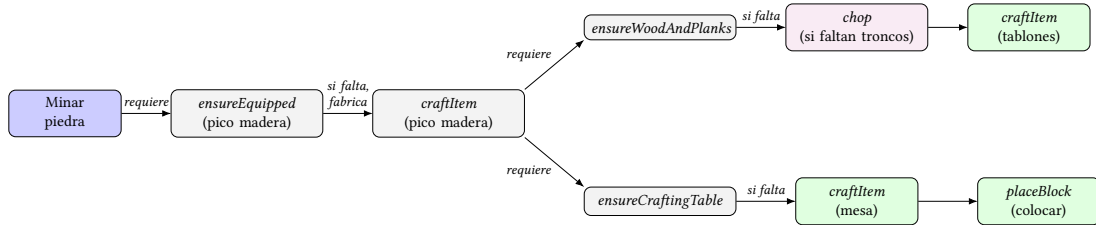


Figura 6.4: Resolución recursiva de precondiciones para minar piedra. Cada tarea garantiza el estado que necesita antes de aplicarse: si una precondición no se cumple, se resuelve como subtarea (en **azul** la tarea objetivo, en **gris** las tareas compuestas, en **verde** las primitivas y en **rosa** la recuperación). El descenso encadena la obtención de madera, la fabricación de tablones y la colocación de la mesa de trabajo, hasta poder fabricar el pico de madera.

6.5.2 Alcance del planificador

El planificador ejecuta la jerarquía de forma determinista: descompone el objetivo en las fases necesarias, resuelve las precondiciones y realiza las acciones sin entrenamiento. Las fases de madera y piedra se ejecutan con fluidez porque los recursos están en la superficie y dentro del *FOV*. La fase de hierro introduce una dificultad adicional: el mineral se genera en torno a $Y = 15-16$ sin una posición (X, Z) conocida de antemano, por lo que localizarlo requiere una exploración subterránea a ciegas que alarga los episodios y los hace poco reproducibles. Por ese motivo en este trabajo no se presenta una métrica de la progresión completa, sino únicamente de la tarea de *woodcutting*.

Iteración II - Agente de Imitación

ESTA iteración aborda el aprendizaje por imitación (IL), entrenando una política que reproduce el comportamiento del experto simbólico a partir del conjunto de demostraciones generado en la iteración anterior (Sección 6.3, página 27).

Para mostrar la evolución del agente, se realiza una explicación de las iteraciones realizadas durante el desarrollo del modelo de imitación: desde una primera versión que aprende solo del estado, pasando por la adición de la visión y la limitación de tratar la cámara como acciones discretas, hasta la arquitectura final de doble cabeza.

Como resultado se obtiene un agente de imitación que consigue realizar la tarea de *wood-cutting*. Sin la información privilegiada de la API, el agente aprende a orientarse hacia el tronco y atacarlo. Si bien no presenta una fase de exploración consistente (Sección 7.7.1, página 42), la política aprendida sí que consigue el objetivo de la tarea bajo condiciones favorables.

7.1 Motivación

A diferencia del agente HTN (Capítulo 6), que decide a partir de la información de la API, el agente de imitación debe aprender a actuar a partir de la información que le presenta el experto. En este caso: la imagen del visor en primera persona y un pequeño vector de estado, con información que el jugador podría obtener fácilmente, como su cambio de posición o si lo que está viendo es un tronco.

El *dataset* utilizado para el entrenamiento consta de un total de 118 episodios y 4 094 pares OBSERVACIÓN-ACCIÓN (4 069 tras la limpieza), cada uno con la estructura descrita en la Tabla 6.2 (página 28). El problema principal encontrado a la hora de realizar el entrenamiento es el propio experto: como ya se vio en la Figura 6.3 (página 28), la distribución de acciones está fuertemente desbalanceada, ya que el experto solo ejecuta unas pocas de las acciones disponibles. Sus causas y su tratamiento se abordan en la Sección 7.2.2 (página 32).

7.2 Entrenamiento sobre el Dataset

En esta sección se describen las decisiones de diseño al procesar el *dataset* de demostraciones y los parámetros utilizados en el entrenamiento.

7.2.1 Fuga de Datos

Como los *frames* de un mismo episodio son muy similares entre sí, una partición aleatoria del *dataset* filtraría información del conjunto de *test* al de entrenamiento, contaminando al modelo. La división del conjunto se hace, entonces, por sesión: episodios completos van a entrenamiento o a *test*, nunca repartidos. Esto garantiza que la validación mide la capacidad de generalizar en episodios no vistos.

7.2.2 Tratamiento del Desbalanceo

Para el tratamiento del desbalanceo de clase observado en el *dataset* (Figura 6.3, página 28), se aplican las tres técnicas mostradas a continuación. Si bien estos tratamientos podrían eliminar información del experto, lo que se intenta con ellos es que el modelo aprenda un comportamiento realista usando como referencia al experto:

- **Pesos por clase balanceados:** las acciones minoritarias pesan más durante el entrenamiento. Dado que la acción dominante *attack* representa el 70 % de los *frames*, su contribución al gradiente se reduce para que el modelo no aprenda a predecirla siempre.
- **Descarte de clases poco representadas:** el HTN navega de forma tan óptima con el *pathfinder* que el *dataset* apenas contiene cuatro de las nueve acciones discretas. El filtro descarta además las clases con menos muestras que un umbral (p. ej., *jump*, con solo 25 apariciones de 4 094 *frames*), que no aportan ejemplos suficientes para un comportamiento real, de modo que el espacio discreto queda reducido a tres clases (*attack*, *move_forward_sprint* y *move_forward_jump*).
- **Aumento de datos (*Data Augmentation*):** las imágenes del entorno son simétricas. Dadas las tres acciones consideradas, su imagen correspondiente es la misma que su versión volteada horizontalmente. Esto lo podemos aprovechar para duplicar el *dataset*: se invierte la imagen del visor, se intercambian las etiquetas *move_left* ↔ *move_right* y se invierte el signo de Δyaw .

7.2.3 Hiperparámetros

Como optimizador se utiliza *AdamW* ($lr=10^{-4}$, $weight_decay=10^{-2}$), un planificador *ReduceLROnPlateau* que reduce el *learning rate* al estancarse la pérdida de validación, y *early stopping* sobre la precisión de validación.

Además, en la normalización de la cámara, los Δyaw y $\Delta pitch$ se acotan a ± 1 rad antes de normalizar, eliminando giros bruscos (> 1 rad) que son generados por el *pathfinder* y que no buscamos que el modelo reproduzca. La Tabla 7.1 (página 33) resume la configuración:

Hiperparámetro	Valor
Longitud de secuencia (T)	4 frames
Optimizador	<i>AdamW</i> ($lr=10^{-4}$, $weight_decay=10^{-2}$)
Planificador de lr	<i>ReduceLROnPlateau</i> (factor 0,5, $min_lr=10^{-6}$)
Pérdida de acción	<i>CrossEntropyLoss</i> (pesos balanceados, <i>label smoothing</i>)
Pérdida de cámara	<i>MSELoss</i>
Tamaño de <i>batch</i>	32
Partición <i>train/val</i>	por sesión (evita fuga temporal)
Regularización	<i>dropout</i> (0,5 acción, 0,3 cámara), <i>weight decay</i> , <i>early stopping</i>
Aumentado	Volteo horizontal con intercambio de etiquetas

Tabla 7.1: Configuración de entrenamiento del agente IL.

7.3 Implementación de Solo Estado

Las primeras versiones del agente intentan simular la percepción de la versión simbólica. Si bien el agente de la primera iteración presenta una imitación de visión usando el FOV, no es una visión real como la que tendría un jugador, con la capacidad de ver literalmente los bloques, árboles, etc. Por ello, la primera iteración utiliza únicamente el vector de estado que, eliminando el componente de visión, es lo más cercano a la información que tendría el experto que podemos obtener.

Con ello se utiliza una **única cabeza de clasificación**, mostrada en la Figura 7.1 (página 34), sobre un espacio de acciones discretas, donde el modelo recibe como entrada únicamente el vector de estado. Esta versión *limitada* permite generar una cota inferior de referencia para próximas versiones. Cualquier modelo visual posterior debe, como mínimo, igualar lo que se obtiene con esta versión simple.

El vector de estado consta de 6 componentes normalizadas a $[-1, 1]$ mediante *cotas fijas* conocidas del entorno¹: orientación de cámara (*yaw*, *pitch*), movimiento respecto al *fra-*

¹ Rangos máximos de la cámara, máxima distancia posible entre posiciones de un *frame* a otro, distancia máxima

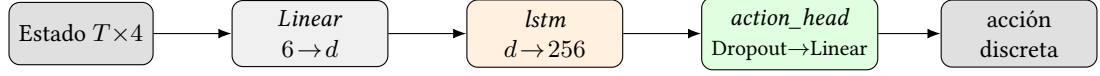


Figura 7.1: Arquitectura de la variante solo estado: el vector de estado sirve de entrada a la red **LSTM** y una única cabeza de clasificación predice la acción discreta. La variable d es la dimensión del espacio de características. Sin entrada visual ni cabeza de cámara, es la base sobre la que las variantes siguientes añaden el extractor visual y la segunda cabeza (Figura 7.3, página 36).

me anterior ($\Delta x, \Delta z$) y dos señales de percepción obtenidas a través del **FOV** (*tree_visible*, *tree_distance*). Se excluye la posición absoluta (x, y, z) ya que con coordenadas absolutas la red puede tender a memorizar posiciones del mundo en lugar de generalizar; el movimiento útil se encuentra en $\Delta x, \Delta z$. La red presenta un componente temporal ($T = 4$ frames), ya que las acciones se benefician de la información contextual a lo largo del tiempo. Si el agente se encuentra atacando, es probable que en el siguiente *frame* siga atacando, o si se encuentra moviéndose hacia el tronco, es probable que siga moviéndose hacia él.

La salida del modelo es, entonces, la mejor acción a realizar dado un entorno. Estas acciones recogen las acciones de movimiento, ataque y ocho acciones de cámara que aplican giros fijos de $\pm 15^\circ$ y $\pm 45^\circ$ en *yaw* y *pitch*, hasta sumar 17 acciones en total.

Observando la Tabla 7.5 (página 42), la variante solo-estado alcanza una tasa de éxito del 4 % (recoger $k = 5$ troncos), con una media (μ) de 0,54 troncos por episodio y una mediana ($\tilde{x}$) de 0,00. Esto indica la incapacidad del agente para aprender a realizar la tarea de forma ciega y muestra la necesidad de incorporar la entrada visual. Se refuerza con la Figura 7.8 (página 42), donde se observa que el agente realiza la acción de ataque el 99,3 % de las veces. Al estar en un entorno rodeado de troncos, el agente aprende a atacar constantemente, pero es incapaz de orientarse o saber si es capaz de romperlos.

7.4 Incorporación de la Entrada Visual

Como se menciona en la sección anterior, en la variante solo-estado el agente es incapaz de orientarse hacia un tronco, ya que desconoce la posición real del mismo. La segunda versión incorpora la **imagen del visor** (Apéndice B) sumada al vector de estado. Esto proporciona al modelo gran parte de la información que un jugador humano tendría, permitiendo al agente aprender a orientarse hacia el tronco y atacarlo, sin cruzar la línea de la omnisciencia, como sería usar el *pathfinder* de la versión simbólica (Capítulo 6). Se ilustra la arquitectura de la variante visual de una cabeza en la Figura 7.2 (página 35).

La imagen de entrada se redimensiona a 128×128 y se normaliza usando los valores de media y desviación típica del *dataset ImageNet*, ya que uno de los extractores visuales

hacia un árbol.

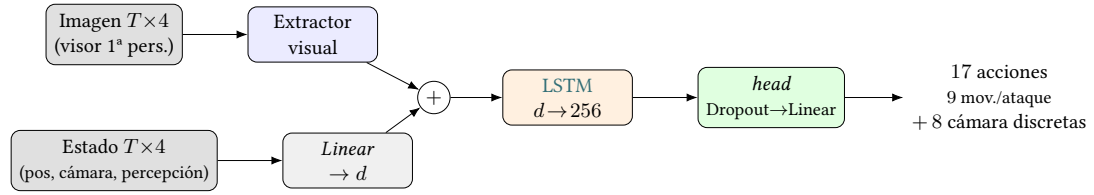


Figura 7.2: Arquitectura de cabeza única: la imagen y el estado se fusionan por suma, pasan por la LSTM y una única cabeza de clasificación elige una de 17 acciones discretas, de las cuales 8 son giros de cámara fijos ($\pm 15^\circ$, $\pm 45^\circ$). Al ser mutuamente excluyentes, el agente no puede girar la cámara y desplazarse en el mismo paso; esta limitación motiva la doble cabeza (Figura 7.3, página 36).

utilizados es un modelo preentrenado con este conjunto de datos [35]. Un **extractor** produce un vector de características por *frame* de la imagen del visor que se fusiona con el estado mediante una suma, cuyo resultado alimenta a la red final LSTM (Figura 7.3, página 36). La salida final es una acción que ejecutará el agente, ya sea moverse, atacar o girar la cámara.

El principal cambio entre las variantes visuales es el extractor, cuya selección final se aborda en la Sección 7.5 (página 37). Cabe mencionar que una de las variantes, aparte de modificar el extractor, cambia la red troncal por una *Convolutional Long Short-Term Memory* (ConvLSTM) en lugar de una LSTM². Para el vector de estado se realiza una proyección lineal a un espacio de características de la misma dimensión que el extractor visual, para permitir la suma de los vectores. A diferencia de la concatenación, la suma mantiene la entrada a la LSTM en un tamaño fijo d independientemente de si hay imagen o no, de modo que las variantes solo-estado y visual comparten exactamente la misma arquitectura. Además, al usar la suma, el tamaño del vector resultante es menor al de la concatenación; provocando una leve mejora en el coste de entrenamiento.

Por último, la red troncal empleada es una LSTM con 256 unidades ocultas, seleccionada por su capacidad de aprender dependencias temporales. Como ya se señaló en la Sección 7.3 (página 33), las acciones no son independientes entre sí, por lo que se mantiene el *frame-stacking* ($T = 4$ frames) para aportar contexto temporal.

7.4.1 Cámara como Acciones Discretas y su Limitación

El modelo simbólico era capaz de realizar múltiples acciones a la vez (moverse/atacar y mover la cámara simultáneamente). Pero con el diseño de una sola cabeza las acciones son mutuamente excluyentes (una por *frame*); mover la cámara y avanzar o atacar no pueden ocurrir a la vez. En la práctica el experto sí ajusta la cámara mientras se acerca al tronco, de modo que la información de cámara se perdía en todos los *frames* con una acción de movimiento o ataque.

² A lo largo del capítulo se referenciará la ConvLSTM como un extractor más por conveniencia.

A esta limitación se añade que las ocho clases de cámara, con apenas 5 a 39 muestras cada una, caían constantemente bajo el umbral de *min_samples* y se eliminaban del entrenamiento. Si el agente mueve la cámara 15° en 5 de cada 4 094 *frames* de entrenamiento, no es un comportamiento a aprender, sino ruido para el modelo.

El modelo de cabeza única alcanzaba una precisión de acción superior (92,5 %), como recoge la Tabla 7.2 (página 36), pero solo porque la tarea quedaba reducida a un problema binario (*attack* frente a *move_forward_sprint*) en el que el agente no aprendía a orientar la cámara en absoluto.

Estas limitaciones motivan la arquitectura final, donde el agente predice la cámara con relación a la acción que tiene que ejecutar, pudiendo dejar la cámara sin mover con un $\Delta yaw = 0$ y $\Delta pitch = 0$.

Modelo	Acc. acción (%)	Clases	Cámara
Cabeza única	92,5	2	8 clases discretas, eliminadas en limpieza
Cabeza dual	77,3	3	Continua (regresión Δyaw , $\Delta pitch$)

Tabla 7.2: Comparación de la cabeza única (cámara discreta) frente a la doble cabeza. La mayor precisión de la cabeza única es engañosa: solo conserva dos clases de acción porque descarta toda la cámara; la doble cabeza conserva las acciones disponibles del experto y, además, modela el movimiento de cámara continuo.

7.4.2 Implementación de la Cabeza Dual

La solución al problema de la cámara consiste en separar el movimiento de cámara del resto de acciones, de modo que la cámara deje de competir con ellas y pase a registrarse y predecirse en cada *frame* como una variable separada. El nuevo espacio de control pasa a ser: una acción discreta y un movimiento de cámara continuo (Δyaw , $\Delta pitch$, resultante del movimiento de un *frame* a otro). La arquitectura resultante se observa en la Figura 7.3 (página 36). A continuación se describen las dos cabezas de la arquitectura final:

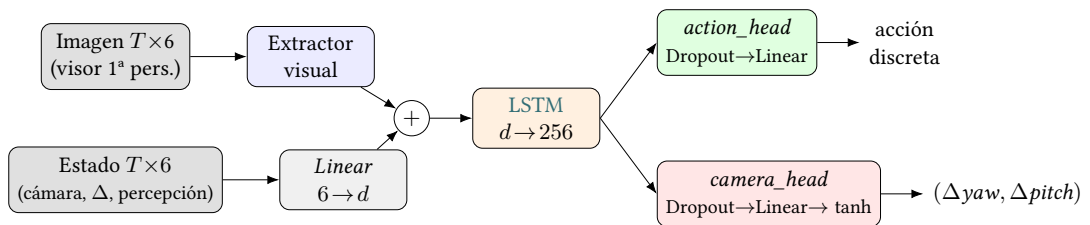


Figura 7.3: Arquitectura final de doble cabeza. Las características visuales y el estado proyectado se suman, pasando a una red LSTM que aporta contexto temporal y dos cabezas de salida para la predicción de: la acción discreta (clasificación) y el desplazamiento de cámara continuo (regresión con tanh).

- **Cabeza de clasificación** (*action_head*): dado el estado y las características visuales, predice la acción correspondiente.
- **Cabeza de regresión** (*camera_head*): predice $(\Delta yaw, \Delta pitch)$ con una activación tanh que acota la salida a $[-1, 1]$, el mismo rango al que se normalizan en el entrenamiento. Luego se desnormaliza a radianes en inferencia para el control del agente y, además, se descartan los giros bruscos (> 1 rad) que el *pathfinder* del HTN introduce de forma puntual.

Cada cabeza tiene su propia función de pérdida; la cabeza de acción utiliza la función *CrossEntropyLoss* con pesos balanceados y *label smoothing*, mientras que la cabeza de cámara usa *MSELoss*. La función de pérdida total de la red se calcula como la suma de ambas pérdidas:

$$\mathcal{L} = \mathcal{L}_{\text{acción}} + \mathcal{L}_{\text{cámara}} \quad (7.1)$$

7.5 Selección del Extractor Visual

Una vez fijada la arquitectura de doble cabeza, es necesario seleccionar el mejor extractor visual para la red. Se evaluaron tres alternativas, recogidas en la Tabla 7.3 (página 38), todas bajo la misma configuración (misma partición de datos, mismos hiperparámetros):

- *Gated Recurrent Unit* (GRU): trata cada fila de la imagen como una secuencia, procesando la imagen de arriba abajo. Se eligió usar una GRU frente a una *Recurrent Neural Network* (Red Neuronal Recurrente) (RNN) clásica tras compararlas en igualdad de condiciones: la GRU alcanzó un 77,3 % frente al 66,9 % de la RNN (10,4 puntos de ventaja).
- *ConvLSTM*: combina una *Convolutional Neural Network* (Red Neuronal Convolutacional) (CNN) con una celda recurrente convolutacional. Con esta arquitectura, se pretendía ver si la parte más significativa de la arquitectura era el extractor visual o la red troncal.
- *Vision Transformer* (ViT): preentrenado en *ImageNet*. El modelo utilizado fue adaptado a entradas de 128×128 (64 *tokens* de 16×16).

Lo primero que se observa en la Tabla 7.3 (página 38) es que la entrada visual mejora la precisión frente al solo-estado, aumentando la precisión de acción en 8 puntos (del 69,4 % al 77,3 %) respecto al mejor modelo, lo que confirma que la imagen aporta información que el vector de estado no captura. Entre los extractores, la GRU obtiene el mejor resultado (77,3 %), por delante de la *ConvLSTM* (74,7 %) y del ViT (73,2 %).

Sin embargo, como se puede observar en la Figura 7.4 (página 38), se aprecia un claro ejemplo de *overfitting*: el modelo alcanza una gran precisión de entrenamiento (~ 97 %) pero

Variante	Acc. validación (%)	Loss acción	MSE cámara
Solo estado	69,4	0,749	0,108 7
Visual (GRU)	77,3	1,193	0,125 1
Visual (ConvLSTM)	74,7	0,839	0,125 7
Visual (ViT)	73,2	0,865	0,132 2

Tabla 7.3: Comparación de los extractores visuales evaluados en la iteración. La variante solo-estado sirve de referencia para la mejora que aporta la entrada visual. Como resultado, la GRU obtiene la mayor precisión de acción (77,3 %) y se selecciona como extractor final.

no generaliza tan bien al conjunto de validación (77,3 %). Esta tendencia se repite en todos los extractores, especialmente en el ViT que, a pesar de ser el más pesado y preentrenado, es el que más sufre este fenómeno. Las curvas de entrenamiento del resto de modelos se pueden ver en el Apéndice D.

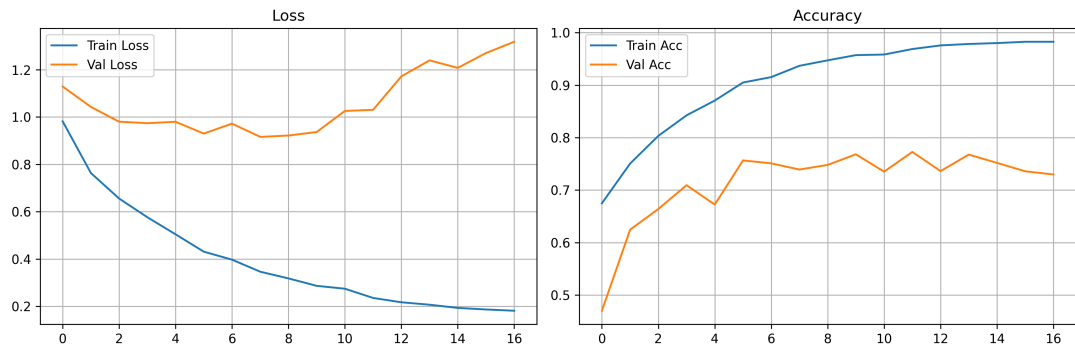


Figura 7.4: Evolución de la precisión y la pérdida (entrenamiento y validación) del modelo GRU de doble cabeza. La precisión de validación alcanza el 77,3 %; el *early stopping* detiene el entrenamiento al estancarse la mejora.

Además, las redes empleadas son considerablemente pequeñas, como muestra la Tabla 7.4 (página 39): la ConvLSTM es la más ligera (0,38 M de parámetros), pero queda 2,6 puntos por debajo de la GRU. El ViT es el más pesado (22,2 M, de los cuales solo quedan 0,63 M entrenables) tras congelar las capas y, sin embargo, es el peor en precisión. La GRU ofrece el mejor compromiso entre coste y precisión: con 3,75 M de parámetros y sin preentrenamiento, alcanza la mayor precisión. Esto la convierte en el modelo ganador de la iteración.

Observando la matriz de confusión (Figura 7.5, página 39) se puede apreciar de dónde provienen los errores del modelo. Las acciones *attack* y *move_forward_sprint* son las que más seguridad tienen, con un 77,0 % y un 91,0 % respectivamente. El principal problema se encuentra en la acción de *move_forward_jump*. Esto se debe a que, para un mismo *frame* y estado, las acciones *move_forward_jump* y *move_forward_sprint* son muy similares. Presentan

Extractor	Parámetros	Preentrenamiento	Acc. val. (%)
GRU	3,75 M	No	77,3
ConvLSTM	0,38 M	No	74,7
ViT	22,2 M (0,63 M entrenables)	Sí (<i>ImageNet</i>)	73,2

Tabla 7.4: Comparación de tamaño de los tres extractores. El ViT mantiene su *backbone* congelado y solo entrena la cabeza de clasificación, mientras que la GRU y la ConvLSTM se entrenan desde cero.

diferencias sutiles, lo que provoca que el modelo no sea capaz de capturar su distinción.

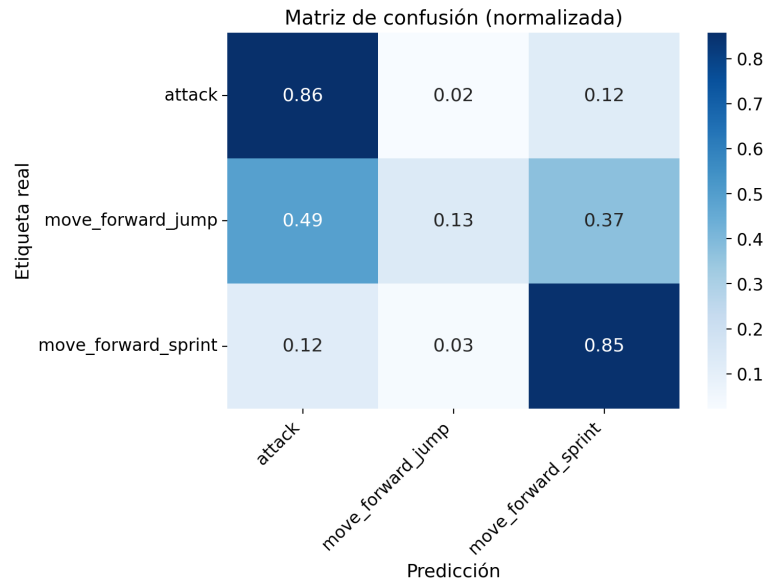


Figura 7.5: Matriz de confusión sobre el conjunto de validación. Los errores se concentran entre las dos acciones de avance; la acción dominante *attack* se clasifica con alta fiabilidad.

En las imágenes del visor (véase el Apéndice B), el agente simbólico podría realizar ambas acciones de movimiento: tanto *move_forward_sprint* como *move_forward_jump* para llegar al mismo resultado. Cabe mencionar que la división de estas dos acciones es deliberada: si solo usásemos las acciones de *jump* y *move_forward_sprint*, el agente sería incapaz de sobrepasar bloques, ya que la acción de *jump* no realiza movimiento horizontal, solo vertical. Lo más importante es que, si reducimos las acciones a atacar y moverse, observamos que el agente sí que aprende los comportamientos de acercarse al tronco y atacarlo (Figura 7.6, página 40). La confusión que podría generarse en este escenario binario viene dada por una razón similar: las acciones de movimiento y de ataque tienen imágenes bastante similares, diferenciándose principalmente por la distancia al tronco y la posición de la cámara.

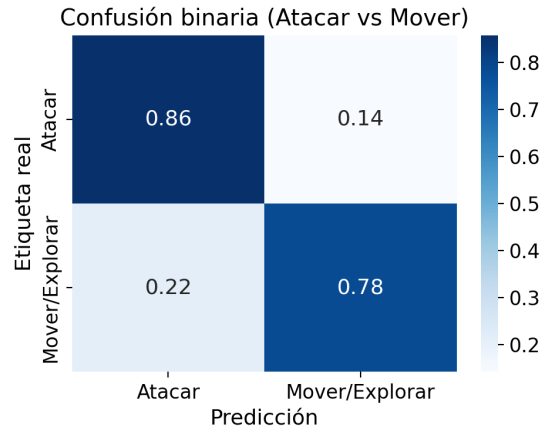


Figura 7.6: Matriz de confusión sobre el conjunto de validación colapsando las dos acciones de avance (*move_forward_sprint* y *move_forward_jump*) en una única clase **Mover/Explorar** frente a **Atacar**. Al eliminar la ambigüedad entre las dos acciones de avance, la precisión sube al 83,0 % (frente al 77,3 % de las tres clases): el agente distingue con fiabilidad cuándo acercarse al tronco (78 %) de cuándo talarlo (86 %).

7.6 Servidor de inferencia

Siguiendo la arquitectura de tres capas del Capítulo 5, el modelo entrenado (*Python*) y el control del agente (*Node.js*) se comunican por **HTTP**: un servidor de inferencia expone el *endpoint* `POST /predict`, que recibe la imagen del visor y el estado del agente y responde con la acción predicha, su confianza y el desplazamiento de cámara desnormalizado (en radianes).

Sobre esta comunicación, el *ILAgent* (capa de agentes) ejecuta un bucle *percepción* → *decisión* → *acción* (Figura 7.7, página 41): captura el *prismarine-viewer* con *Puppeteer*, lo envía al servidor junto con el estado actual y ejecuta en *Mineflayer* la acción que recibe.

Se aplican tres matices a la evaluación para adaptar la política a la ejecución en tiempo real:

- **Umbral de confianza:** solo se cambia de acción si la confianza de la nueva supera un umbral (0,7); en caso contrario se mantiene la acción anterior. Esto reduce la ambigüedad entre estados similares, produciendo un comportamiento más estable.
- **Zona muerta de cámara:** los desplazamientos de cámara por debajo de $\sim 0,01$ rad se ignoran por considerarse ruido.
- **Decaimiento del *pitch*:** tras aplicar el delta, el *pitch* se reduce para contrarrestar la acumulación de pequeños errores que, sin corrección, provocarían que el agente apuntara la cámara al suelo o al cielo.

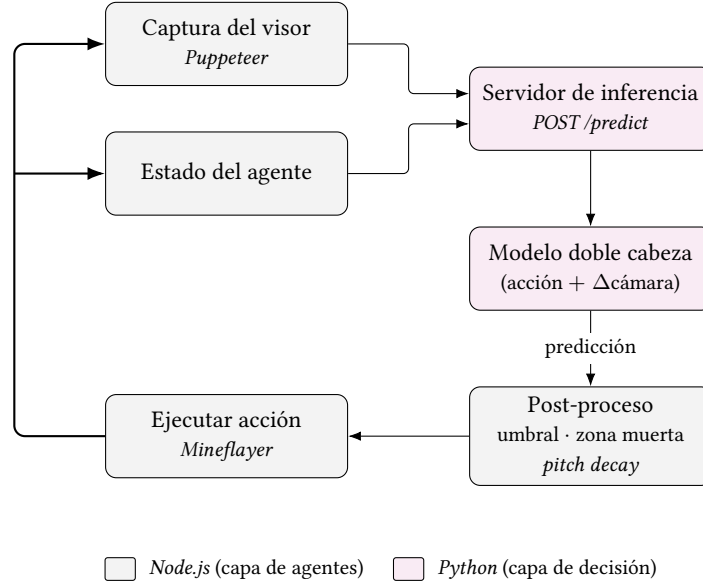


Figura 7.7: Bucle *percepción* $\rightarrow$ *decisión* $\rightarrow$ *acción* del *ILAgent*. La captura del visor y el vector de estado (*Node.js*) se envían al servidor de inferencia (*Python*), que devuelve la acción discreta y el desplazamiento de cámara; la predicción se post-procesa (umbral de confianza, zona muerta de cámara y decaimiento de *pitch*) y se ejecuta en *Mineflayer*, cerrando el bucle en tiempo real.

Además, la acción discreta *attack* se ejecuta como la primitiva de minar del *HTN* (una vez comenzada se realiza hasta romper el bloque), garantizando que la acción de atacar complete la tala del bloque, igual que lo haría el experto.

7.7 Resultados de la iteración

El agente de imitación se evalúa en el mismo entorno y bajo la misma métrica común que el resto de técnicas (recoger al menos k troncos, con $k = 5$). Esto permite comparar al aprendiz con su tutor en igualdad de condiciones. En la Figura 7.8 (página 42) se observa la distribución de acciones del agente de imitación.

En ella podemos apreciar que las dos arquitecturas con una distribución de acciones más similar a la del experto son la *GRU* y la *ViT*, con distribuciones similares entre sí. Las otras dos aproximaciones, la *ConvLSTM* y la variante solo-estado, presentan una distribución de acciones muy característica: ambas realizan la acción de atacar en casi todos los *frames* y apenas se mueven. Esto indica, como se menciona en la Sección 7.3 (página 33), que la información visual obtenida del extractor es la que permite al agente aprender a moverse y atacar. Esto queda confirmado al ver el rendimiento de la *ConvLSTM* en la Tabla 7.5 (página 42), ya que su extractor visual es una *CNN* simple, obteniendo un éxito inferior incluso al de la variante solo-estado, que no tiene información visual. Esto indica que la *CNN* no es capaz de extraer

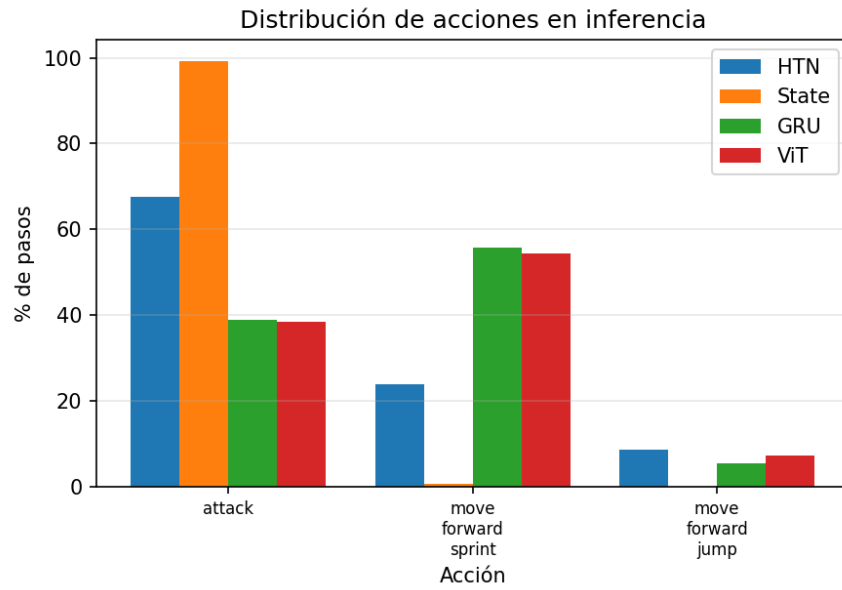


Figura 7.8: Distribución de acciones del agente de imitación por técnica. La GRU y la ViT presentan una distribución de acciones más similar a la del experto (HTN), mientras que la ConvLSTM y la variante solo-estado apenas se mueven y atacan en casi todos los frames.

correctamente la información de la imagen. E incluso puede llegar a reducir el funcionamiento del modelo.

Técnica	Eps.	Éxito (%)	Troncos			Tiempo (s)		
			μ	$\tilde{x}$	σ	μ	$\tilde{x}$	σ
HTN	50	100,0	6,00	6,00	0,00	26,7	26,4	1,1
IL-STATE	50	4,0	0,54	0,00	1,25	195,8	202,1	34,1
IL-GRU	50	52,0	3,84	5,00	1,82	156,2	203,4	67,3
IL-CONV	50	0,0	0,08	0,00	0,27	203,0	202,1	3,3
IL-ViT	32	37,5	3,00	3,50	1,90	169,4	208,6	67,9

Tabla 7.5: Evaluación de las técnicas de imitación bajo la métrica común recoger $k = 5$ troncos. Troncos y tiempo por episodio: media (μ), mediana ($\tilde{x}$) y desviación típica (σ).

7.7.1 Análisis de resultados

Durante el proceso de evaluación se encontró una limitación del modelo de imitación que no se refleja en los resultados mencionados hasta ahora. El agente es capaz de aprender la tarea de talar un árbol visible; es decir, cuando hay un tronco dentro del campo de visión y al alcance, se orienta hacia él, se acerca y lo ataca correctamente.

Sin embargo, el agente **no presenta exploración**: es incapaz de buscar consistentemente

un árbol que esté lejos y acercarse a él. En escenarios donde el árbol no está en el campo de visión inicial, el agente tiene un rendimiento considerablemente peor.

La causa de esta separación viene de las demostraciones generadas por el experto. Al localizar los troncos con la información de la *API* y navegar de forma óptima con el *pathfinder*, nunca realiza la tarea de exploración ni realiza una búsqueda visual. En consecuencia, el *dataset* no contiene comportamiento exploratorio real: el aprendiz solo ha visto cómo talar un árbol que tiene enfrente, pero no cómo encontrarlo mediante exploración. Teniendo esto en cuenta, todas las evaluaciones de las técnicas se realizan en un entorno con árboles al alcance del campo de visión inicial.

7.7.2 Alcance del agente de imitación

La falta de exploración descrita es un síntoma de un problema más general de la imitación. Cuando el agente aprendido se desvía de la trayectoria del experto, visita estados poco representados en las demostraciones, donde la política no es tan precisa. Este problema se denomina *distribution shift* [15]: los errores se acumulan a lo largo del episodio porque el experto no enseñó cómo salir de las situaciones que provoca el propio aprendiz.

Esta limitación es precisamente lo que motiva la Iteración III, desarrollada en el Capítulo 8. El aprendizaje por refuerzo permite que el agente *explore* y corrija sus propios errores mediante la recompensa, en lugar de depender de un experto que cubra todas las posibilidades. Además, la política de imitación puede servir para inicializar el agente de refuerzo mediante *BC*, como se explora en el Capítulo 10.

Iteración III - Agente de Refuerzo

LA última iteración aborda el aprendizaje por refuerzo (RL). A diferencia del agente de imitación (Capítulo 7), que clona a un experto heredando también sus defectos (Sección 7.7.1, página 42), el agente de refuerzo aprende su política interactuando con el entorno y corrigiendo sus propios errores a partir de una señal de recompensa. Teóricamente esto le permite explorar y descubrir comportamientos que el *dataset* del experto no contiene.

Para abordar el aprendizaje por refuerzo se comparan tres algoritmos usados en la literatura: DQN [3], PPO [24] y SAC [25]. Como en las iteraciones anteriores, la tarea es *woodcutting* y el entorno es el mismo, con la misma percepción y las mismas restricciones de acción. El objetivo es estudiar si el refuerzo, aprendiendo su propia política, es capaz de igualar o superar al resto de las técnicas.

8.1 Motivación

El agente HTN (Capítulo 6) decide a partir de la información privilegiada de la API y navega de forma óptima con el *pathfinder*; el agente de imitación (Capítulo 7) reproduce ese comportamiento, pero solo aprende a talar árboles visibles, no a encontrarlos, debido a la navegación óptima del experto.

El aprendizaje por refuerzo pretende suplir estas limitaciones (conocimiento privilegiado y ausencia de exploración), dejando además la posibilidad de descubrir estrategias que el *dataset* del experto no contiene, por ejemplo, evadir los obstáculos. Los fundamentos del refuerzo se describen en el Capítulo 3. Sus mayores limitaciones son: definir una función de recompensa que codifique el objetivo de la tarea (Sección 8.2.3, página 46) y el tiempo de exploración del entorno por parte del agente (Tabla 8.3, página 53).

8.2 Formulación del Problema de Refuerzo

El bucle de aprendizaje que realiza el agente consiste en: observar el estado del entorno, ejecutar una acción y recibir una recompensa, repitiendo hasta una condición de **terminación** o **truncamiento**. En las siguientes secciones se describen los elementos que definen el problema de refuerzo.

8.2.1 Espacio de Observación

La observación combina, como en el agente de imitación, una **imagen** en primera persona y un **vector de estado**, de modo que el agente percibe la misma información que tendría un jugador humano.

El vector de estado consta de 8 componentes normalizadas a $[-1, 1]$ mediante cotas fijas conocidas del entorno: orientación de cámara (*yaw*, *pitch*), movimiento respecto al *frame* anterior (Δx , Δz), dos señales de percepción obtenidas a través del FOV (*tree_visible*, *tree_distance*) y dos señales de progreso de la tarea; *log_count*, número de troncos en el inventario, e *is_looking_at_log*, si el cursor apunta a un tronco. Como en la imitación, se excluye la posición absoluta (x, y, z) para no memorizar posiciones del mundo, el movimiento útil viene de $\Delta x, \Delta z$.

En comparación con el agente de imitación, se añaden dos elementos al vector de estado: *log_count* informa al agente del progreso en el episodio y *is_looking_at_log* le ayuda a identificar cuándo está alineado con un tronco. Ambos elementos son conocidos por un jugador, ya que puede acceder a la información de su inventario para saber cuántos troncos lleva, y mediante la vista en primera persona puede saber si está apuntando a un tronco o no. Estos no se utilizan en **IL** ya que van implícitos en la información que otorga el experto, si realiza la acción de ataque es porque está apuntando a un tronco.

La imagen de entrada se redimensiona a 128×128 . Dado que la red no es recurrente (Sección 8.3, página 47), la información temporal se aporta mediante *frame-stacking*: se apilan los 4 últimos fotogramas en el eje de canales ($4 \times 3 = 12$ canales de entrada). Esto permite a la red inferir el movimiento, que una sola imagen no captura. Es la técnica utilizada por las **DQN** de *Atari* [3], el mismo mecanismo se aplica al vector de estado.

8.2.2 Espacio de Acción

El agente dispone de siete acciones discretas: *attack*, *move_forward_sprint*, *move_forward_jump* y cuatro acciones de cámara (*camera_left*, *camera_right*, *camera_up*, *camera_down*). Las acciones de cámara aplican giros fijos de 0,15 rad ($\sim 8,6^\circ$) en *yaw* y 0,10 rad en *pitch*.

A este espacio de acciones no se añaden todas las opciones de movimiento, ya que como se observa en la Figura 6.3 (página 28), el agente experto es capaz de talar un árbol sin necesidad

de desplazarse lateralmente ni hacia atrás. Por lo que se considera que el espacio de acciones es suficiente para la tarea.

A diferencia del agente de imitación, en refuerzo la cámara se controla como una acción más. En consecuencia, igual que ocurría en la primera versión de cabeza única del agente de imitación (Sección 7.4.1, página 35), mover la cámara y desplazarse vuelven a ser mutuamente excluyentes. Esta decisión de diseño se toma porque el agente de refuerzo no tiene que copiar a un experto que realice múltiples acciones en un mismo paso, sino que aprende por sí mismo cuándo conviene mirar y cuándo desplazarse.

8.2.3 Función de Recompensa

La **función de recompensa** es la señal numérica que indica al agente cómo de bien o mal lo está haciendo en cada momento; es el único canal por el que el refuerzo aprende, y su diseño determina cómo de bueno será el rendimiento del agente. Esta señal se compone de tres términos:

- **Recompensa terminal:** una bonificación grande al cumplir la condición de éxito del episodio. En este caso romper y recoger al menos un tronco.
- **Recompensas intermedias** (*reward shaping*): señales extra que guían la exploración hacia el objetivo (golpear un tronco, acercarse a un árbol visible). Sin ellas, la recompensa sería demasiado escasa y el agente apenas recibiría información de cómo realizar la tarea.
- **Penalización por paso:** un coste pequeño en cada paso que penaliza comportamientos en bucle y favorece iteraciones eficientes.

En este caso, la métrica de éxito de un episodio es romper y recoger al menos **un** tronco. A diferencia del agente simbólico, que tiene como objetivo k troncos. La retropropagación de la recompensa para talar k troncos en un solo episodio podría ser demasiado escasa, por lo que se opta por una recompensa más corta. Con esto se busca que el agente aprenda a talar un tronco, y que en la evaluación final (Capítulo 9) se pueda expandir su rendimiento en la métrica de k troncos.

Los valores empleados para el diseño de la función de recompensa se recogen en la Tabla 8.1 (página 47). El valor de acercarse a un árbol se calcula como la diferencia de distancia al árbol más cercano entre el paso actual y el anterior. Esta información no se considera omnisciente ya que, aunque un jugador no pueda medir la distancia exacta a un árbol, sí puede percibir si se acerca o se aleja de él X bloques.

Las recompensas positivas (romper un tronco, recogerlo y el éxito del episodio) son mucho más grandes que las intermedias y las negativas. Esto se hace para que el agente aprenda a talar

Evento	Valor	Propósito
Romper un tronco	+20,0	Objetivo principal
Recoger un tronco	+5,0	Señal principal de progreso
Golpear un árbol	+1,0	Señal auxiliar de alineación
Acercarse a un árbol	+0,1	Señal de acercamiento
Éxito del episodio	+30,0	Bonificación terminal
Coste por paso	-0,05	Favorecer trayectorias cortas
Atacar un bloque erróneo	-1,0	Penalizar romper bloques incorrectos
Terminación anticipada	-5,0	Penalización por muerte

Tabla 8.1: Términos de la función de recompensa. Los valores positivos son mucho mayores que los negativos para favorecer el aprendizaje de la tarea principal.

y recoger troncos, y no se quede en un comportamiento de acercarse a un árbol o golpearlo sin romperlo. Además, dado el grado de libertad del agente, buscamos acotar el espacio de exploración y guiarlo hacia el objetivo de la tarea lo máximo posible, que es talar y recoger troncos.

8.2.4 Condiciones de Terminación

Un episodio puede finalizar de cuatro formas:

- **Éxito:** el agente recoge al menos un tronco y lo ha roto él mismo en el episodio. Esta segunda condición es una salvaguardia que evita que el agente recoja la madera caída de un episodio previo y la obtenga sin talar, inflando artificialmente la tasa de éxito. La evaluación final (Capítulo 9) equipara el umbral (k troncos) al resto de las técnicas.
- **Muerte:** el agente muere durante el episodio.
- **Colapso de recompensa:** si la recompensa acumulada cae por debajo de un umbral (-10), el episodio se corta.
- **Truncamiento:** al alcanzar el horizonte máximo de pasos ($MAX_STEPS=100$ en entrenamiento).

8.3 Arquitectura de la Red

Los tres algoritmos comparten el mismo **extractor de características** (visual y de estado); lo único que cambia es la cabeza específica de cada algoritmo. Así se garantiza que las

diferencias de rendimiento se deban al algoritmo de aprendizaje y no a la capacidad de extracción de la red. La arquitectura mostrada en la Figura 8.1 (página 48) procesa la imagen apilada con un extractor CNN, la fusiona con el vector de estado igual que en la imitación, y sobre él cada algoritmo aplica su propia cabeza.

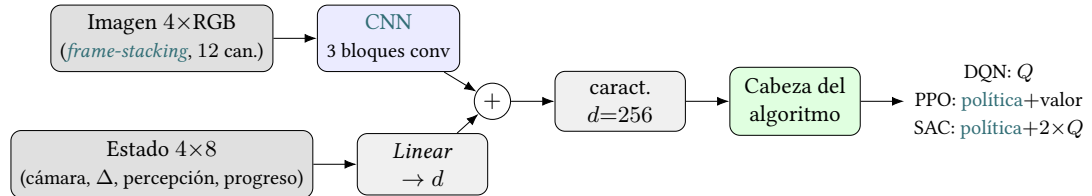


Figura 8.1: Extractor de características común a los tres algoritmos y cabeza específica de cada uno. La imagen apilada (4 *frames*, 12 canales) pasa por un extractor CNN; las características visuales (dim. d) se suman al estado proyectado, formando un vector de características común. Sobre él, cada algoritmo aplica su cabeza: valores Q en DQN; actor y crítico en PPO; actor y dos críticos (*twin* Q) en SAC. La temporalidad se aporta por apilado de fotogramas, no por recurrencia.

Si bien la arquitectura es similar a la del agente de imitación (Sección 7.4.2, página 36), existen matices que impiden su transferencia directa:

- **Criterio de selección distinto:** la GRU se seleccionó por ser la que mejor rendimiento obtuvo en un *dataset* supervisado, estático, pequeño y desbalanceado de demostraciones del experto. El refuerzo optimiza otro objetivo (valores Q o política) sobre datos no estacionarios que el propio agente genera; ser el mejor en imitación no implica serlo en refuerzo.
- **Temporalidad:** en la imitación la dependencia temporal la aportaba la LSTM sobre $T = 4$ frames, no el extractor. Aquí esa información la da el *frame-stacking* de la entrada (Sección 8.2.1, página 45), usado en la DQN de Atari [3]. La CNN sobre frames apilados ya permite capturar el movimiento.

El extractor visual es una CNN ligera de tres bloques convolucionales (canales $12 \rightarrow 32 \rightarrow 64 \rightarrow 128$), cada uno con normalización y *max-pooling*, que reduce la imagen de 128×128 a 16×16 ; el resultado se aplanar y se proyecta a un vector de características de dimensión 256. El vector de estado se proyecta linealmente a esa misma dimensión y se fusiona por suma con las características visuales, igual que en la imitación. Hasta aquí, la red es idéntica en los tres algoritmos.

Sobre ese vector de características final, cada algoritmo añade su propia **cabeza** que produce su salida característica:

- **DQN:** una cabeza que estima los valores Q de las siete acciones.

- **PPO**: dos cabezas, un *actor* (distribución sobre las acciones) y un *crítico* (valor del estado).
- **SAC**: una cabeza de *actor* (*política*) y dos críticos Q (*twin Q*).

En el diseño además se incluyó el uso de *GroupNorm* en lugar de *BatchNorm*. Los algoritmos basados en valor mantienen dos copias de la red (red en línea y red objetivo); *BatchNorm* mantiene estadísticas globales que son distintas entre las redes y pueden desincronizar sus salidas para la misma entrada. *GroupNorm* no mantiene estadísticas globales y evita esa alteración [36].

8.4 Algoritmos de refuerzo

Se comparan tres algoritmos, cada uno representativo de una familia distinta del refuerzo: **DQN** [3] (basado en valor), **PPO** [24] (gradiente de *política on-policy*) y **SAC** [25] (actor-crítico *off-policy*).

8.4.1 Deep Q-Network (DQN)

El **DQN** [3] aprende una función de valor-acción $Q(s, a)$ que estima la recompensa esperada de tomar la acción a en el estado s ; la *política* elige la acción de mayor valor. Es un método *off-policy*; las transiciones se almacenan en un *replay buffer*, lo que permite reutilizar la experiencia varias veces.

Se emplea la variante **Double-DQN** [37], que separa la selección de la acción (con la red en línea) de la evaluación de su valor (con la red objetivo) para mitigar la sobreestimación de los valores Q . La exploración sigue una estrategia ϵ -greedy con *decaimiento a lo largo del entrenamiento*: al principio el agente actúa casi al azar y, conforme avanza, confía cada vez más en su *política*.

8.4.2 Proximal Policy Optimization (PPO)

El **PPO** [24] optimiza directamente la *política* mediante ascenso de gradiente, sin usar una función de valor-acción. Es un método *on-policy*: aprende de la misma *política* que genera los datos, por lo que recolecta un conjunto de su experiencia, realiza épocas de actualización sobre él y lo descarta. No reutiliza experiencia antigua.

Presenta un objetivo recortado (*clipped objective*), que limita cuánto puede cambiar la *política* en cada actualización y evita pasos exagerados. Sigue un esquema *actor-crítico*: el *actor* produce la distribución sobre acciones y el *crítico* estima el valor del estado.

8.4.3 Soft Actor-Critic (SAC)

El SAC [25] es *actor-crítico* como PPO pero *off-policy* como DQN, de modo que aprende una *política* reutilizando un *replay buffer*. Se caracteriza por utilizar **máxima entropía**: además de maximizar la recompensa, favorece que la *política* mantenga la mayor entropía posible, lo que fomenta una mejor exploración.

Se emplea la variante discreta del algoritmo, adaptada al espacio de siete acciones de la tarea. Mantiene dos críticos Q para reducir la sobreestimación al igual que la DQN, el peso de la entropía se ajusta automáticamente durante el entrenamiento, evitando tener que fijarlo a mano.

8.5 Configuración de Entrenamiento

8.5.1 Integración con el Entorno

Como el agente de imitación, el refuerzo sigue la arquitectura de tres capas (Capítulo 5) y captura de la imagen con *Puppeteer*. La diferencia está en el mecanismo de control: en la imitación el agente consulta a un servidor de inferencia; en refuerzo es el entorno *Gymnasium* el que dirige el bucle y envía las acciones al *bridge* mediante */step* y */reset*, recibiendo el estado, los eventos del agente (bloques rotos, troncos recogidos...) y la recompensa (Figura 8.2, página 51).

Además, */world_reset* regenera el mundo con la misma *semilla* entre episodios, algo que no ocurría en la imitación al no tener un entrenamiento directo sobre el entorno.

8.5.2 Mundo de Entrenamiento

El entrenamiento se realiza en un *bioma* de bosque generado con una *semilla* fija, de modo que el escenario sea consistente entre episodios. Para evitar que el agente sobreaprenda un único estado del mundo (árboles ya talados, madera en el suelo), el mundo se regenera periódicamente cada 10 episodios. En la comparación de los algoritmos de refuerzo se emplea un punto de aparición fijo distinto del de entrenamiento, para medir la *política* en condiciones no vistas.

8.5.3 Hiperparámetros

La Tabla 8.2 (página 52) resume la configuración común y la específica de cada algoritmo. Todos comparten la misma arquitectura de red, factor de descuento ($\gamma = 0,99$), tamaño de imagen (128×128), *frame-stacking* ($k = 4$) y horizonte de episodio (100 pasos). El valor alto de γ se utiliza ya que la mayor recompensa llega al final del episodio (romper y recoger el

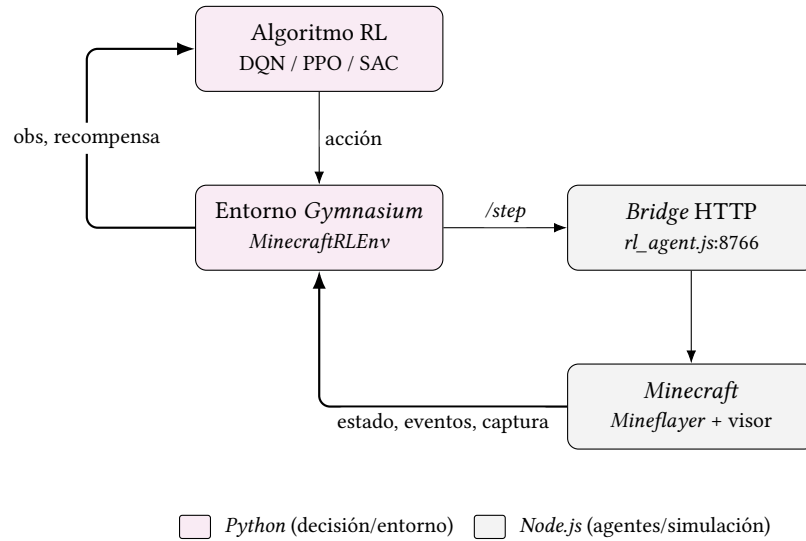


Figura 8.2: Bucle de entrenamiento. El entorno *Gymnasium* envía una acción al *bridge* (*/step*); el agente la ejecuta en *Minecraft* y devuelve: el estado, los eventos y la captura del visor, a partir de los cuales el entorno calcula la observación y la recompensa.

tronco), y nos interesa que esa señal se propague hacia atrás para considerar la secuencia de acciones que la hizo posible.

8.5.4 Selección del *Checkpoint*

Durante el entrenamiento se guarda un *checkpoint* cada 10 episodios. Como el aprendizaje por refuerzo puede sufrir una reducción de rendimiento por estados no beneficiosos (puntos de aparición donde el agente no pueda encontrar árboles, exploraciones irrelevantes), el *checkpoint* final no es necesariamente el mejor. Para cada algoritmo se selecciona el *checkpoint* correspondiente al **pico de la media móvil de la tasa de éxito** (ventana de 20 episodios) calculada sobre las métricas de entrenamiento, a diferencia de la **media global** que contaría estos episodios ruidosos; la media móvil evalúa el rendimiento del modelo en un momento seleccionado. El mismo criterio se aplica a los tres algoritmos.

El mejor *checkpoint* de cada algoritmo según este criterio es: *Double-DQN* en el episodio 130, PPO en el episodio 160 y SAC en el episodio 50. El agente aleatorio, al no entrenar, no tiene *checkpoint* y se evalúa tal cual. Estas son las versiones que se comparan a continuación y, en el Capítulo 9, frente al resto de las técnicas.

Algoritmo	Hiperparámetro	Valor
Común	Descuento γ	0,99
Común	Dim. características / oculta	256 / 256
Común	Imagen / <i>frame-stacking</i>	$128 \times 128 / k = 4$
Común	Horizonte (pasos)	100
Común	Tamaño de <i>batch</i>	64
DQN	Optimizador / <i>lr</i>	<i>Adam</i> / 10^{-4}
DQN	<i>Replay buffer</i> / <i>warmup</i>	1×10^5 / 3 000
DQN	Decaimiento ε	10 000 pasos
DQN	Actualización red objetivo	cada 2 500 pasos
DQN	<i>Reward clip</i> / <i>grad clip</i>	1,0 / 10,0
PPO	Optimizador / <i>lr</i>	<i>Adam</i> / 3×10^{-4}
PPO	<i>Rollout</i> / épocas / minibatch	1 024 / 4 / 64
PPO	Recorte / <i>Generalized Advantage Estimation</i> (GAE) λ	0,2 / 0,95
PPO	Coef. entropía / valor	0,01 / 0,5
SAC	<i>lr</i> (actor/crítico/ α)	3×10^{-4}
SAC	<i>Replay buffer</i> / <i>warmup</i>	1×10^5 / 1 000
SAC	Soft update τ	0,005
SAC	Entropía objetivo	98 % (auto α)

Tabla 8.2: Configuración de entrenamiento de los algoritmos de refuerzo.

8.6 Resultados de la Iteración

Esta sección compara los tres algoritmos y una implementación aleatoria, que sirve de *línea base*: los algoritmos que no lo superen no están aprendiendo nada útil o precisan de más tiempo de entrenamiento. Las métricas utilizadas en esta sección son: las curvas de entrenamiento con la media móvil (Sección 8.5.4, página 51), los resultados de evaluación en un mundo con punto de aparición fijo y la métrica común de k troncos.

8.6.1 Curvas de aprendizaje y comparativa global

Las Figuras 8.3 (página 54) y 8.4 (página 54) muestran la tasa de éxito y la recompensa por episodio (media móvil, ventana de 20) de los cuatro agentes. La línea continua marca el progreso hasta el mejor *checkpoint* y la discontinua el resto del entrenamiento. La Tabla 8.3 (página 53) muestra sus valores en el pico y la Tabla 8.4 (página 53) su rendimiento en validación.

Algoritmo	Episodios	Éxito (%)	Recompensa	Tiempo (h)
<i>RANDOM</i>	200	35,0	18,04	2,0
DQN	200	40,0	27,48	3,7
PPO	200	65,0	32,75	2,2
SAC	200	40,0	23,14	3,4

Tabla 8.3: Comparativa de algoritmos de refuerzo durante el entrenamiento. El éxito y la recompensa se miden en el pico de la media móvil (ventana de 20 episodios), es decir, en el *checkpoint* seleccionado para cada algoritmo.

Técnica	Eps.	Éxito (%)	Troncos			Tiempo (s)		
			μ	$\tilde{x}$	σ	μ	$\tilde{x}$	σ
HTN	50	100,0	6,00	6,00	0,00	26,7	26,4	1,1
<i>RANDOM</i>	50	6,0	1,50	1,00	1,52	97,6	111,1	40,3
DQN	50	2,0	1,10	0,00	1,40	117,6	121,5	28,8
PPO	50	12,0	2,14	2,00	1,64	114,6	124,5	43,3
SAC	50	4,0	1,62	1,00	1,41	110,6	114,3	31,3

Tabla 8.4: Evaluación de las técnicas de refuerzo bajo la métrica común recoger $k = 5$ troncos. Troncos y tiempo por episodio: media (μ), mediana ($\tilde{x}$), y desviación típica (σ).

El algoritmo con mejor rendimiento es **PPO**, alcanza la mayor tasa de éxito en el pico (65 %) y la mayor recompensa (32,75), y su curva mantiene una tendencia ascendente, lo que sugiere que con más episodios podría seguir mejorando. **DQN** y **SAC** quedan por detrás, ambos rondando el 40 % de éxito en el pico; su tendencia indica un deterioro, aunque para confirmarlo

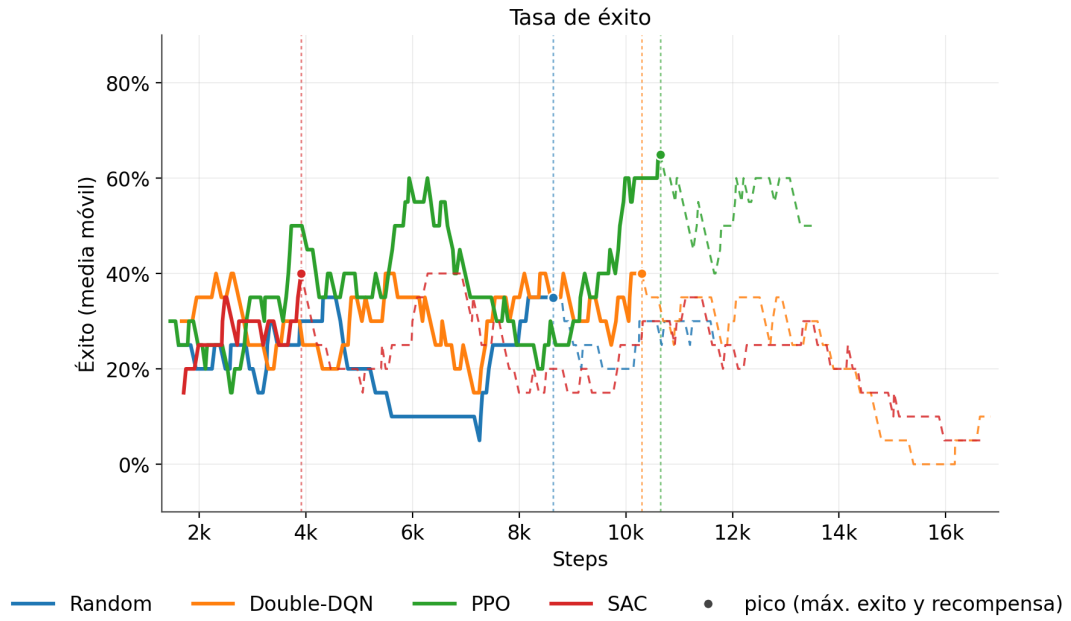


Figura 8.3: Tasa de éxito durante el entrenamiento (media móvil, ventana de 20 episodios). PPO obtiene el mejor rendimiento, seguido de DQN y SAC; los tres superan al agente aleatorio. La línea continua llega hasta el mejor *checkpoint* y la discontinua hasta el final del entrenamiento.

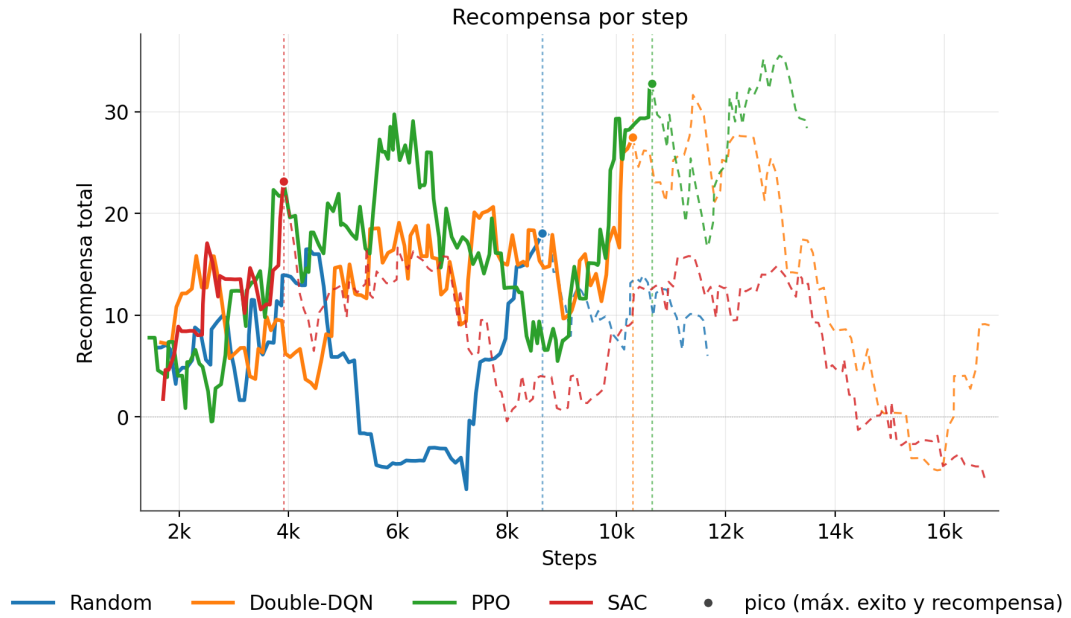


Figura 8.4: Recompensa total por episodio durante el entrenamiento (media móvil, ventana de 20 episodios). El orden coincide con el de la tasa de éxito. La caída final de SAC por debajo de cero es coherente con su inestabilidad (Figura 8.5, página 55).

sería necesario un mayor número de episodios para observar realmente si se estabilizan o siguen cayendo. Aun así, los tres algoritmos superan a la [línea base](#) aleatoria (35 %, 18,04 de recompensa), lo que confirma que sí hay aprendizaje.

Conviene matizar que las caídas observadas no son principalmente un problema de exploración, sino la dificultad de la tarea combinada con el diseño del mundo ya que el punto de aparición se mantiene fijo durante 10 episodios (Sección 8.5, página 50). Si en ese punto no hay árboles cercanos, el agente cae en un agujero o queda atrapado, encadena varios episodios sin éxito que hunden la media móvil, independientemente de si el agente aprendió a evitar esas situaciones o no. Esto explica por qué la curva final (línea discontinua) no se estabiliza o continúa descendiendo, y refuerza la elección del pico de la media móvil frente al [checkpoint](#) final.

8.6.2 Estabilidad del entrenamiento

La diferencia entre PPO y los otros dos algoritmos no solo está en la tasa de éxito, sino en la estabilidad del proceso. La Figura 8.5 (página 55) muestra dos señales propias de los algoritmos que pueden ayudar a explicar el deterioro de [DQN](#) y [SAC](#).

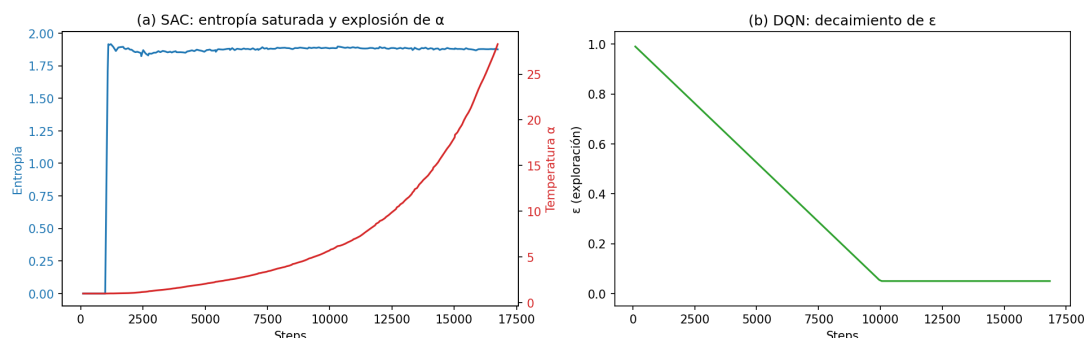


Figura 8.5: Señales internas del entrenamiento. **(a)**: en SAC, la temperatura α (rojo) se dispara mientras la entropía de la [política](#) (azul) permanece cerca de su máximo, indicando que el término de entropía domina y la [política](#) nunca se vuelve decisiva. **(b)**: en DQN, el decaimiento programado de ϵ funciona correctamente. Pero la red podría aprovecharse de un mayor periodo de exploración.

En [SAC](#) (a), la temperatura de entrenamiento (α) se ajusta de forma automática (Sección 8.4.3, página 50). En la práctica α no se estabiliza, sino que crece sin control (de ~ 1 a ~ 28 al final del entrenamiento), de forma que la entropía termina dominando a la recompensa, y el agente no llega a fijar una [política](#) concreta. Su entropía acaba llegando a ≈ 2 rápidamente, indicando que la [política](#) es casi uniforme y el agente actúa de forma aleatoria. Esto corresponde con la caída de su recompensa por debajo de cero (Figura 8.4, página 54) y su distribución de acciones (Figura 8.6, página 56). Se probaron distintos valores de entropía

objetivo (-1 , -2 y -3), pero en todos los casos α continúa creciendo sin control y la política no se vuelve decisiva. No se descarta el rendimiento de SAC, pero su aplicabilidad requiere un ajuste más fino de hiperparámetros y un mayor número de episodios de entrenamiento.

En DQN (b) la exploración la controla ϵ , que decae de forma uniforme de 1,0 a 0,05 a lo largo de los primeros $\sim 10\,000$ pasos. El mejor rendimiento obtenido por la DQN se obtiene alrededor de los $\sim 10\,000$ pasos. El punto donde la red comienza su explotación es visible en la Figura 8.3 (página 54). Con un mayor periodo de exploración, la DQN podría haber alcanzado un mejor rendimiento al poder aprender mejor las variaciones del entorno.

8.6.3 Distribución de acciones

La Figura 8.6 (página 56) muestra la distribución de cada una de las siete acciones durante la evaluación de los cuatro agentes.

El agente aleatorio reparte las acciones de forma uniforme. SAC es muy similar a la versión aleatoria, lo que confirma el análisis de la sección anterior: la explosión de α lo deja en una política casi aleatoria. DQN sí muestra una distribución de aprendizaje; gran parte de las acciones realizadas son *camera_down*, seguido de *camera_up*. Esto corresponde con el proceso de tala que podría realizar un jugador humano; al talar el tronco que tiene a la altura de los ojos, la red aprende a talar los troncos adyacentes en el eje vertical. PPO, en cambio, es el único que reparte el peso entre las acciones más relevantes de la tarea: *attack*, *move_forward_sprint* y *move_forward_jump*, que es justo la combinación de avanzar hacia un árbol y golpearlo que sigue el experto.

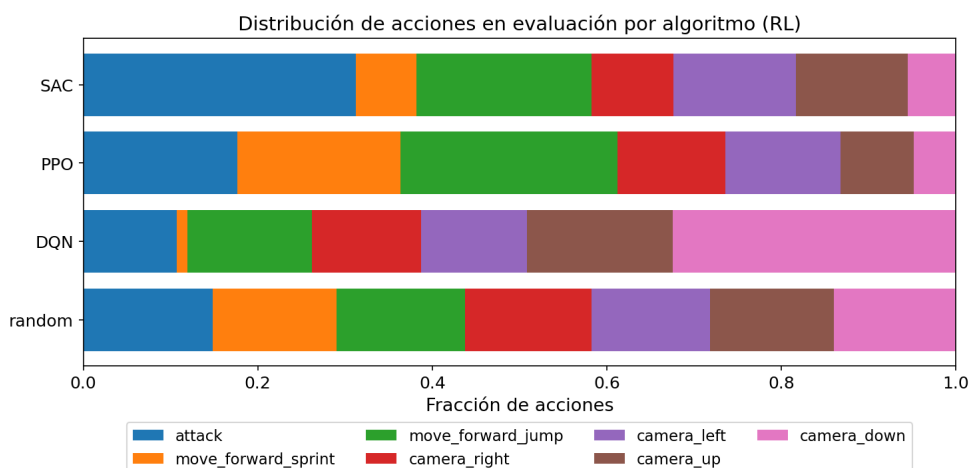


Figura 8.6: Fracción de cada acción durante la evaluación. SAC y el agente aleatorio reparten las acciones de forma casi uniforme; DQN se concentra en *camera_down*; PPO favorece las acciones productivas de la tarea (*attack* y movimiento).

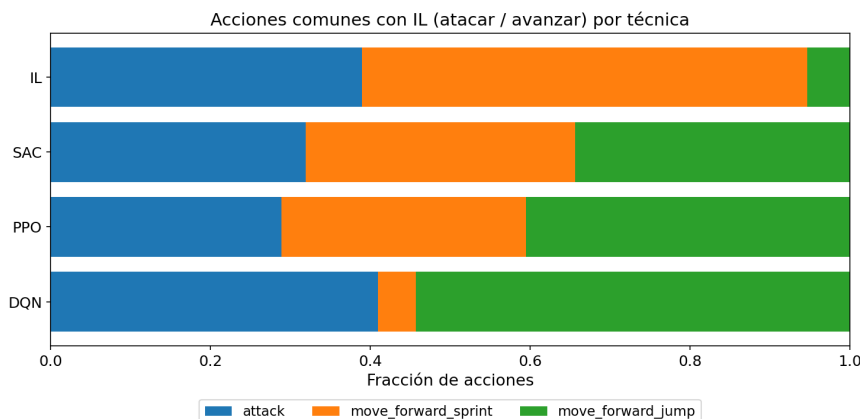


Figura 8.7: Fracción de cada acción durante la evaluación, eliminando las acciones que no están en el espacio de acciones del agente de imitación.

Además, la Figura 8.7 (página 57) muestra la distribución de acciones de los tres algoritmos de refuerzo, eliminando las acciones que no están en el espacio de acciones de imitación (Sección 7.4.2, página 36). Es decir, eliminando el componente de la cámara, para ver si los algoritmos de refuerzo aprenden el mismo tipo de distribución que la imitación del experto. El más cercano a la distribución es curiosamente el **DQN**, que aunque presenta un éxito menor al de PPO, sí aprende a moverse hacia el árbol y golpearlo. Aunque con las acciones de movimiento intercambiadas en relación al **IL**, el *move_forward_sprint* en **DQN** tiene la misma distribución que *move_forward_jump* en **IL**, y viceversa.

8.6.4 Discusión

El refuerzo parece ser la técnica con un mayor horizonte de desarrollo, pero en consecuencia presenta el mayor coste computacional y temporal. La literatura (Capítulo 2) muestra que el refuerzo puede aprender tareas de *Minecraft* como la tala de árboles, o la obtención de diamante [5]. Pero a coste de un inmenso conjunto de datos y tiempo de entrenamiento. El número de episodios de entrenamiento utilizados en este trabajo es muy bajo en comparación con otros trabajos. Por ejemplo, los agentes de *MineRL* [5] emplean para la misma tarea de tala unos 1 500 episodios (del orden de 12 millones de pasos), lo que en nuestro entorno equivaldría a unas 1 800 horas de cómputo (unos 75 días continuos) por algoritmo.

Los resultados de esta iteración muestran que el refuerzo es capaz de aprender la tarea, aunque sin llegar a un rendimiento comparable al de la aproximación simbólica o de imitación. Pero también muestran que el refuerzo puede no requerir de un conjunto de datos y tiempo de cómputo tan grande como el de *MineRL* [5]. Con un conjunto de datos de 200 episodios (unas 2 horas de cómputo) es posible obtener un rendimiento aceptable que muestre tendencias hacia un rendimiento óptimo.

Discusión de Resultados

ESTE capítulo realiza una comparación de todas las técnicas implementadas a lo largo del trabajo: HTN (Capítulo 6), IL (Capítulo 7) y RL (Capítulo 8). Todas ellas entrenadas y evaluadas sobre la misma tarea y bajo una métrica común de tasa de éxito: talar $k = 5$ troncos.

Si bien en cada capítulo se analizan en profundidad los resultados de cada implementación, este capítulo pretende generar un análisis más general comparando todas las familias.

9.1 Metodología de Evaluación

Todas las técnicas se evalúan bajo las mismas condiciones para que la comparación sea justa. Cada agente ejecuta 50 **episodios** en el mismo mundo, con un **punto de aparición fijo** y distinto del usado en el entrenamiento, de modo que ninguna técnica evalúe sobre un entorno que haya visto. Entre episodios el mundo se reinicia al mismo estado con la misma **semilla**, de modo que todas las técnicas se enfrentan al mismo entorno. Cada episodio se trunca a un máximo de 200 pasos para contabilizar la eficiencia. La semilla concreta no es relevante, cualquier mundo con un árbol en el campo de visión inicial es válido, ya que el objetivo es evaluar la habilidad de talar y recoger, no la búsqueda mediante exploración ciega.

La métrica de éxito es común a todos los algoritmos; un episodio se considera exitoso si el agente *rompe y obtiene al menos $k = 5$ troncos*. Esta métrica se utiliza para mostrar las limitaciones de los algoritmos; el HTN siempre va a recoger el tronco que tala, pero el IL, al igual que en la exploración, es posible que no recoja los troncos. Lo mismo ocurre en RL, aunque en menor medida. Se exige recoger y no solo romper porque el fin de talar es obtener el recurso, que un agente rompa un tronco sin recogerlo no es el objetivo de la tarea.

9.2 Resultados por Familia

Junto a la tasa de éxito se reportan métricas auxiliares por episodio: troncos recogidos (media (μ), mediana ($\tilde{x}$) y desviación típica (σ)) y tiempo empleado. La Tabla 9.1 (página 59) reúne la tasa de éxito, los troncos recogidos y el tiempo por episodio de cada técnica. La Figura 9.1 (página 59) ofrece la comparación visual de la tasa de éxito por técnica.

Técnica	Eps.	Éxito (%)	Troncos			Tiempo (s)		
			μ	$\tilde{x}$	σ	μ	$\tilde{x}$	σ
HTN	50	100,0	6,00	6,00	0,00	26,7	26,4	1,1
RL-RANDOM	50	6,0	1,50	1,00	1,52	97,6	111,1	40,3
RL-DQN	50	2,0	1,10	0,00	1,40	117,6	121,5	28,8
RL-PPO	50	12,0	2,14	2,00	1,64	114,6	124,5	43,3
RL-SAC	50	4,0	1,62	1,00	1,41	110,6	114,3	31,3
IL-STATE	50	4,0	0,54	0,00	1,25	195,8	202,1	34,1
IL-CONV	50	0,0	0,08	0,00	0,27	203,0	202,1	3,3
IL-GRU	50	52,0	3,84	5,00	1,82	156,2	203,4	67,3
IL-VIT	50	46,0	3,48	4,00	2,03	156,7	206,1	70,9

Tabla 9.1: Evaluación de las técnicas bajo la métrica común recoger $k = 5$ troncos. Troncos y tiempo por episodio: media (μ), mediana ($\tilde{x}$), y desviación típica (σ).

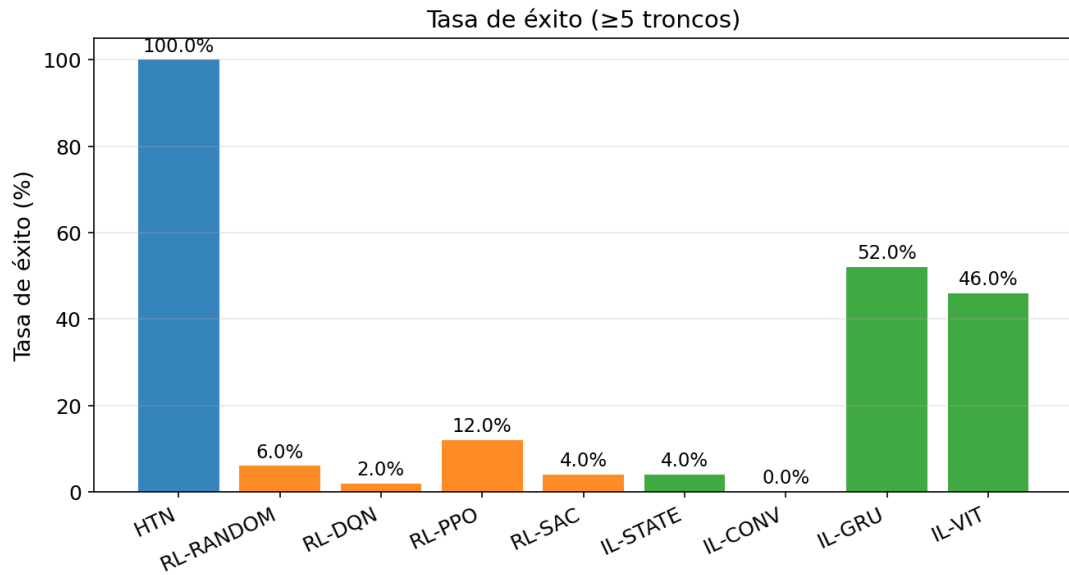


Figura 9.1: Tasa de éxito en entorno bajo la métrica común (≥ 5 troncos, 50 episodios). El HTN alcanza el objetivo el 100 % de las veces; entre los agentes entrenados, IL-GRU (52 %) e IL-VIT (46 %) destacan claramente sobre el refuerzo y el agente aleatorio.

9.2.1 Agente Experto

El HTN obtiene un 100 % **de éxito**, recogiendo exactamente 6 troncos por episodio en 26,7 s de media. El tronco de más sobre el objetivo ($k = 5$) se debe a que el agente tala el árbol completo antes de reevaluar la condición de parada: la primitiva de tala rompe todo el tronco de una vez y, en el mundo de evaluación (*semilla* fija), ese árbol tiene 6 bloques. De ahí también su desviación típica nula. Este resultado era esperable: el planificador opera sobre la información de la API y navega de forma óptima con el *pathfinder* (Capítulo 6). Su tasa de éxito y eficacia lo convierten en la cota superior de la tarea y el límite a alcanzar por el resto de técnicas.

9.2.2 Aprendizaje por Imitación

La IL-GRU es la mejor técnica de todos los agentes entrenados, con un 52 % **de éxito** y una mediana de 5 troncos. A diferencia del HTN, el objetivo se comprueba una vez por paso de inferencia, de modo que entre comprobaciones el agente puede recoger varios troncos a la vez; por eso sus episodios exitosos terminan con 5, 6 o 7 troncos (Figura 9.1, página 59). La variante ViT llega a un 46 %, mientras que las variantes más simples no consiguen un rendimiento apreciable: solo-estado se queda en el 4 % y ConvLSTM en el 0 %. Esto confirma la importancia de la extracción de información de la imagen en la tarea de imitación.

Como se menciona en la Sección 7.7.1 (página 42), el agente aprende a talar un árbol visible, pero no a encontrarlo. De ahí que falle cerca de la mitad de los episodios, o bien porque no tiene un árbol delante o porque se pasa de largo y no lo detecta.

9.2.3 Aprendizaje por Refuerzo

El refuerzo desde cero queda muy por debajo respecto a las técnicas de imitación. El mejor algoritmo es PPO con un 12 % **de éxito**, seguido del agente aleatorio (6 %), SAC (4 %) y DQN (2 %). El dato más importante es que DQN y SAC no superan al *línea base* aleatorio, indicando que no llegan a consolidar una política útil. Solo PPO se separa del *línea base*. Sin embargo, cabe mencionar que sí se observan comportamientos de un funcionamiento correcto, lo que indica que con más episodios (tomando como referencia *MineRL* [5]) probablemente se obtendría un rendimiento mucho mejor.

9.3 Comparativa Global

Ordenando las técnicas bajo la métrica común se obtiene la siguiente jerarquía:

HTN (100 %) $\gg$ IL-GRU (52 %) $\approx$ IL-ViT (46 %) $\gg$ PPO (12 %) $>$ aleatorio (6 %) $>$ SAC, DQN

Usando esta jerarquía y los resultados mencionados anteriormente, podemos concluir la importancia que presenta la información del experto a la hora de completar la tarea y la capacidad de extraer información de las imágenes. Esta premisa se ve claramente con el rendimiento de las técnicas de imitación, frente a las de refuerzo, cuadruplicando la tasa de éxito de su contrapartida, gracias a la información que le proporciona el experto.

Sin embargo, esto no inhabilita la capacidad de aprendizaje del refuerzo, que sí consigue aprender relativamente la tarea. Durante las ejecuciones de entrenamiento y evaluación sí se observa un comportamiento de identificación del árbol, navegación hacia él y tala del mismo. En el Capítulo 10 se proponen unas líneas de trabajo que combinan la información del experto con el refuerzo, para mejorar su rendimiento y eficiencia.

Por último, destacar que, aunque el escenario de evaluación es favorable, los parámetros de evaluación son exigentes. Con un máximo de 200 pasos por episodio, el agente tiene que ser eficiente en su comportamiento y conseguir talar múltiples troncos. Reduciendo el número de troncos objetivo, como se observa en la Figura 9.2 (página 61), los resultados mejoran considerablemente, aunque manteniendo la jerarquía de rendimiento mencionada anteriormente.

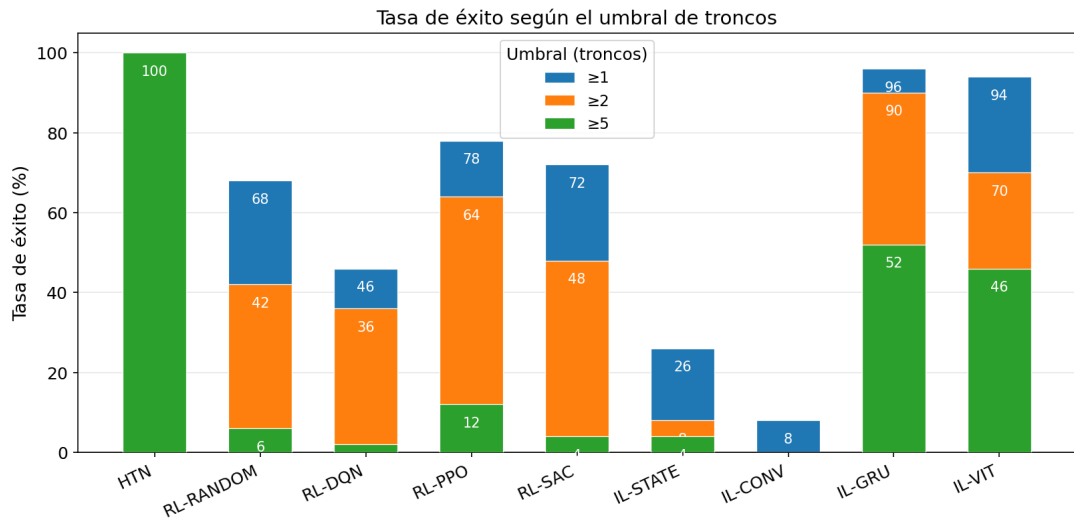


Figura 9.2: Tasa de éxito de cada técnica al exigir ≥ 1 , ≥ 2 y ≥ 5 troncos, superpuestas sobre la misma barra (50 episodios). El HTN mantiene el 100 % en los tres umbrales; en el resto el éxito decae al subir la exigencia, de forma acusada en el refuerzo (PPO: 78 $\rightarrow$ 64 $\rightarrow$ 12 %) y más contenida en IL-GRU (96 $\rightarrow$ 90 $\rightarrow$ 52 %).

Conclusiones

Los objetivos planteados al comienzo del trabajo se han cumplido satisfactoriamente. El punto de partida fue diseñar una arquitectura común que permitiese a los agentes compartir entorno, observaciones y protocolo de evaluación, combinando *Node.js* para el control de bots con *Mineflayer* y *Python* para los modelos de IA. Además, fue necesario tratar la integración entre el servidor *Java* de *Minecraft*, *Node.js* y *Python*, que fue una fuente constante de problemas de sincronización, aunque la arquitectura final es estable y funcional. Otra de las dificultades fue la captura de imágenes en tiempo real y tener que almacenarlas junto al vector de estado al que pertenecían, ya que la obtención de las imágenes y el vector de estado no estaban sincronizados de base.

Sobre este *framework* se implementaron los tres agentes: el planificador HTN (Capítulo 6), los agentes de imitación (Capítulo 7) y los agentes de refuerzo con tres algoritmos distintos (DQN, PPO y SAC) (Capítulo 8).

Para que el aprendizaje por imitación fuese posible fue necesario crear un conjunto de datos propio, recogiendo las ejecuciones del planificador HTN en vivo. La comparación entre técnicas exigió definir una evaluación rigurosa y reproducible (Sección 9.1, página 58): misma semilla, punto de aparición fijo, número de episodios y una métrica de éxito común (≥ 5 troncos talados y recogidos), preservando la restricción de no omnisciencia: imagen en primera persona y estado restringido al FOV, sin acceso a posiciones exactas ni visión a través de obstáculos.

Los resultados, recogidos en el Capítulo 9, permiten extraer varias conclusiones. La principal es la importancia del conocimiento experto; las dos mejores técnicas (HTN e IL) son precisamente las que lo utilizan. Conviene mencionar que la superioridad del HTN es intrínseca a este tipo de tareas; al operar sobre conocimiento codificado a mano, siempre va a realizar la tarea de la mejor forma posible. Sin embargo, esa misma dependencia es su límite, ya que su extensión a jerarquías mucho más complejas no es sencilla y exige un esfuerzo de modelado que crece con la complejidad del dominio. El IL reduce el coste de entrenamiento

respecto al refuerzo y obtiene un mejor rendimiento, ya que aprende de un experto y puede enfocarse en la explotación de la tarea en lugar de explorar desde cero. El **RL**, sin embargo, es capaz de descubrir comportamientos que el experto no enseñó: en la **DQN**, el agente aprende a talar bloques adyacentes (superior e inferior respecto al bloque talado) que el **IL** no presenta, y consigue una navegación más adaptada a su percepción y espacio de acciones. Esta capacidad de generalización tiene el coste de requerir un presupuesto de cómputo mayor: el espacio de acciones con temporalidad fija y la recompensa escasa y diferida limitaron el número de interacciones posibles, evidenciando que la cantidad de recursos es un factor determinante para esta familia de técnicas. Como se menciona en el Capítulo 2, la mayoría de trabajos relacionados utilizan presupuestos de cómputo y datos de magnitudes mucho mayores; bajo un escenario ideal, los agentes dispondrían de un horizonte de rendimiento significativamente más amplio.

El trabajo utiliza muchas de las competencias enseñadas en la titulación: ingeniería de software, desde el desarrollo con *Scrum* hasta el uso de **HTTP** para la comunicación entre modelos; análisis y manejo de datos, con la creación del *dataset* y la evaluación de los agentes; e inteligencia artificial, desde algoritmos con enfoques clásicos como **HTN** hasta aproximaciones profundas como **RL** e **IL**. Además, incorpora herramientas fuera del plan de estudios, como *Node.js*, servidores *Java* o entornos de simulación *3D*, necesarias para llevar el trabajo a cabo, lo que refleja la capacidad de investigar y adoptar nuevas tecnologías para la realización de proyectos.

10.1 Trabajo Futuro

Las líneas de trabajo más prometedoras aprovechan la complementariedad entre los tres paradigmas. Por ejemplo, combinar el **HTN** como tutor de exploración en un bucle **RL** tipo **Dagger** [15] permitiría guiar al agente de refuerzo con conocimiento experto y acelerar significativamente su convergencia. En paralelo, ampliar el *dataset* de demostración y explorar otras arquitecturas para el **IL** podría extender sus capacidades. A más largo plazo, el *framework* desarrollado ofrece una base para escalar la comparación a tareas de mayor complejidad jerárquica dentro de *Minecraft*, como la obtención de diamante o la construcción de estructuras.

Apéndices

Jerarquía de un Pico de Hierro

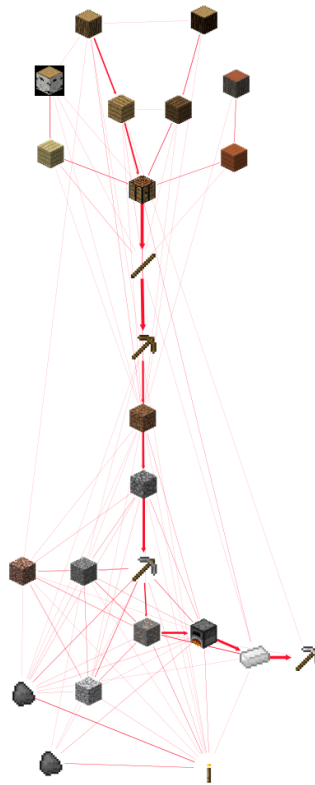


Figura A.1: Jerarquía de dependencias para obtener un pico de hierro en *Minecraft*. Se pueden observar las dependencias necesarias: *Pico de Madera* → *Pico de Piedra* → *Pico de Hierro*. Adaptada de *MineRL* [5].

Frame del dataset de demostraciones

EN este apéndice se muestra un ejemplo real de un *frame* del conjunto de demostraciones generado por el agente simbólico (Capítulo 6). Cada *frame* es un par *OBSERVACIÓN-ACCIÓN*: la imagen del visor en primera persona (Figura B.1, página 66) junto con el registro JSON de su estado, su acción y los metadatos de percepción (Listado B.1, página 67).



Figura B.1: Imagen del visor en primera persona asociada al *frame* del Listado B.1 (página 67): el agente apunta a un tronco situado a 2,7 bloques y ejecuta la acción `attack`.

```
1 {
2   "image":
3     "data/recordings/Rec_100_ablgs0_screenshots/frame_00003.png",
4   "state": {
5     "x": 11.499,
6     "y": 123,
7     "z": 6.581,
8     "yaw": -3.1416,
9     "pitch": -0.343
10  },
11  "action": "attack",
12  "camera_delta": {
13    "dyaw": 0,
14    "dpitch": 0
15  },
16  "session_id": "Rec_100_ablgs0",
17  "tree_visible": true,
18  "tree_distance": 2.7
19 }
```

Listing B.1: Registro JSON de un *frame* (par *OBSERVACIÓN-ACCIÓN*) del dataset de demostraciones.

En este *frame*, el agente tiene un tronco visible a 2,7 bloques (`tree_visible = true`) y ejecuta la acción `attack` sin desplazamiento de cámara (`camera_delta` nulo), correspondiente al instante en que tala el árbol que tiene enfrente.

Árbol de Tareas para la Obtención de un Pico de Hierro

EN este apéndice se muestra la descomposición completa de tareas HTN para la obtención de un pico de hierro en *Minecraft*, mostrando la jerarquía de tareas y subtareas que componen cada fase de la progresión (madera, piedra e hierro). Así como las tareas de recuperación.

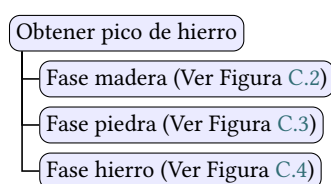


Figura C.1: Árbol de descomposición general de la progresión hasta el pico de hierro. Las subtareas detalladas de cada fase se desglosan en figuras posteriores.

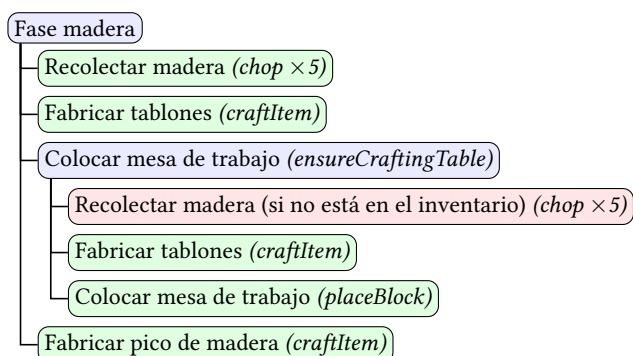


Figura C.2: Detalle del árbol de descomposición para la Fase madera. Esta fase inicia la progresión y reutiliza primitivas y tareas de recuperación como el *woodcutting*.

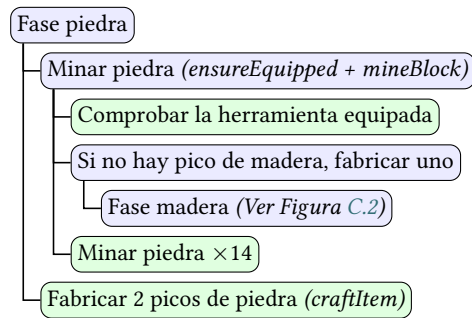


Figura C.3: Detalle del árbol de descomposición para la Fase piedra. Si no se dispone de la herramienta adecuada, se invoca la Fase madera para su creación.

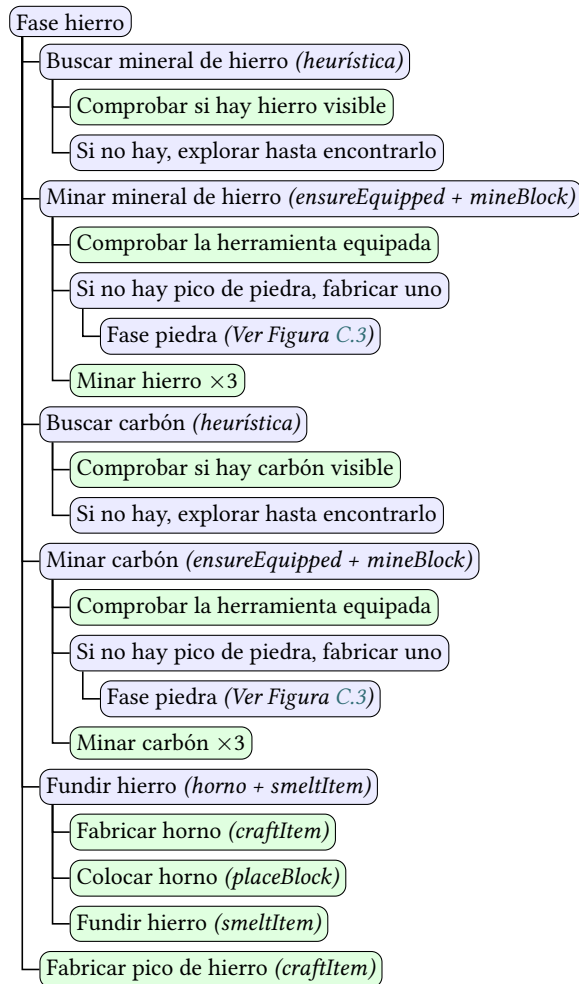


Figura C.4: Detalle del árbol de descomposición para la Fase hierro, incluyendo la búsqueda heurística de minerales y el proceso de fundición.

Variantes de imitación

ESTE apéndice recoge las curvas de entrenamiento de las cuatro variantes comparadas en la Iteración II (Capítulo 7) en la Tabla 7.3 (página 38), todas bajo la misma configuración (igual partición e hiperparámetros, variando solo el extractor). Se muestran juntas para ilustrar el sobreajuste que exhiben los modelos dado el tamaño limitado del dataset de demostraciones y la similitud entre sus clases, siendo la variante ViT el caso más pronunciado.

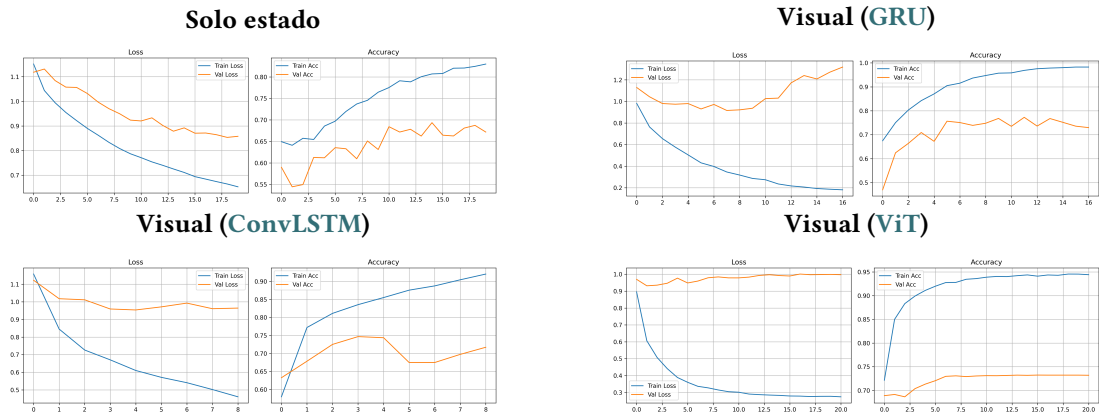


Figura D.1: Curvas de entrenamiento (precisión y pérdida en entrenamiento y validación) de las cuatro variantes bajo configuración idéntica. El modelo de solo estado no alcanza una precisión de validación satisfactoria (cota inferior). Las variantes GRU y ConvLSTM generalizan de forma razonable, con una brecha entre entrenamiento y validación contenida. La variante ViT presenta el sobreajuste más pronunciado: su precisión de entrenamiento alcanza el ~95% mientras la de validación se estanca en torno al 72–73%, una brecha de más de 20% que evidencia que su mayor capacidad no se traduce en mejor generalización con un dataset de este tamaño.

Lista de acrónimos

- API** *Application Programming Interface* (Interfaz de Programación de Aplicaciones). 18, 22, 25, 28, 31, 43, 44, 60, 70, 73
- BC** *Behavioral Cloning*. 9, 43, 70
- CNN** *Convolutional Neural Network* (Red Neuronal Convolucional). 37, 41, 48, 70
- ConvLSTM** *Convolutional Long Short-Term Memory*. 35, 37–39, 41, 42, 60, 70
- CPU** *Central Processing Unit* (Unidad Central de Procesamiento). 14, 70
- Dagger** *Dataset Aggregation*. 5, 9, 63, 70
- DQN** *Deep Q-Network*. 4, 9, 11, 44, 45, 48–50, 52, 53, 55–57, 60, 62, 63, 70
- DRL** *Deep Reinforcement Learning* (Aprendizaje por Refuerzo Profundo). 4, 10, 70
- FOV** *Field of View* (Campo de Visión). 23–26, 28, 30, 33, 34, 45, 62, 70
- GAE** *Generalized Advantage Estimation*. 52, 70
- GPU** *Graphics Processing Unit* (Unidad de Procesamiento Gráfico). 14, 70
- GRU** *Gated Recurrent Unit*. 37–39, 41, 42, 48, 70
- HTN** *Hierarchical Task Network* (Red de Tareas Jerárquicas). 2, 3, 5, 7, 8, 12, 13, 15, 18, 19, 24, 27–29, 31, 32, 37, 41, 42, 44, 58–63, 68, 70
- HTTP** *Hypertext Transfer Protocol* (Protocolo de Transferencia de Hipertexto). 13, 18–20, 40, 63, 70
- HUD** *Interfaz de Información en Pantalla, Heads-Up Display*. 20, 70

IA *Inteligencia Artificial*. 1–5, 62, 70

IL *Imitation Learning* (Aprendizaje por Imitación). 2, 3, 5, 7–9, 13, 18, 31, 33, 45, 57, 58, 62, 63, 70

JSON *JavaScript Object Notation*. 19, 70

LLM *Large Language Model* (Modelo de Lenguaje Grande). 5, 6, 14, 70

MAS *Multi-Agent System* (Sistema Multiagente). 5, 70

MDP *Markov Decision Process* (Proceso de Decisión de Markov). 9, 70

PPO *Proximal Policy Optimization*. 11, 44, 48–50, 52, 53, 56, 62, 70

RAM *Random Access Memory* (Memoria de Acceso Aleatorio). 14, 70

RGB *Red, Green, Blue* (Rojo, Verde, Azul). 21, 70

RL *Reinforcement Learning* (Aprendizaje por Refuerzo). 2–5, 7, 9, 18, 44, 58, 63, 70

RNN *Recurrent Neural Network* (Red Neuronal Recurrente). 37, 70

SAC *Soft Actor-Critic*. 8, 11, 44, 48–50, 52, 53, 55, 56, 60, 62, 70

SAT *Boolean Satisfiability Problem* (Problema de Satisfacibilidad Booleana). 7, 70

STRIPS *Stanford Research Institute Problem Solver*. 7, 70

ViT *Vision Transformer*. 37–39, 41, 42, 60, 70

VPT *Video PreTraining*. 5, 6, 70

YOLO *You Only Look Once*. 14, 70

Glosario

BatchNorm Normalización que reescala las activaciones de una capa usando la media y la varianza de cada *batch*, lo que estabiliza y acelera el entrenamiento. 49, 70

GroupNorm Normalización que divide los canales en grupos y calcula la media y la varianza dentro de cada grupo, de forma independiente del tamaño del *batch*. 49, 70

Gymnasium Librería estándar de *Python* para desarrollar, simular y comparar algoritmos de aprendizaje por refuerzo. 13, 15, 18, 19, 50, 51, 70

Transformer Arquitectura de red neuronal basada en mecanismos de atención. Diseñada para el procesamiento del lenguaje natural, aplicada actualmente a muchos otros dominios. 5, 70

backbone Red troncal de un modelo de aprendizaje profundo, encargada de la extracción inicial de características antes de las cabezas específicas de cada tarea. 39, 70

checkpoint Punto de control en el que se guarda el estado completo del modelo (pesos, parámetros y estado del optimizador) para evaluarlo o reanudarlo más tarde. 51, 55, 70

distribution shift Caída de rendimiento que se produce cuando los datos que el modelo encuentra en inferencia difieren de los que vio durante el entrenamiento. 5, 9, 43, 70

early stopping Técnica de regularización que detiene el entrenamiento cuando el rendimiento en validación deja de mejorar, evitando el sobreajuste. 70

endpoint Punto de acceso (una URL concreta de una [API](#)) a través del cual un cliente se comunica con un servicio para enviar o recibir información. 19, 40, 70

frame-stacking Apilamiento de fotogramas, que agrupa las últimas observaciones consecutivas del entorno para aportar información temporal al agente. 21, 35, 45, 48, 50, 52, 70

- framework** Estructura base de herramientas y convenciones sobre la que se desarrolla un *software*. 13–16, 62, 63, 70
- headless** Modo de ejecución de un *software* o servidor en segundo plano, sin interfaz gráfica de usuario. 17, 70
- max-pooling** Operación de submuestreo en redes convolucionales que reduce la dimensión espacial conservando solo el valor máximo de cada región. 48, 70
- offline** Algoritmo que se entrena sobre un conjunto de datos fijo, recopilado de antemano, sin interactuar con el entorno durante el aprendizaje. 9, 70
- overfitting** Sobreajuste de un modelo, que memoriza los datos de entrenamiento pero pierde capacidad de generalizar ante datos nuevos. 37, 70
- pathfinder** Módulo encargado de buscar rutas óptimas (en este trabajo, con el algoritmo A*) para que el agente navegue sorteando obstáculos. 25, 26, 28, 32–34, 37, 43, 44, 60, 70
- raycast** Trazado de rayos empleado en gráficos 3D y videojuegos para detectar colisiones o líneas de visión, lanzando un rayo virtual desde un punto en una dirección dada. 23, 70
- replay buffer** Memoria que almacena experiencias pasadas del agente para reutilizarlas en las actualizaciones. Propia de los algoritmos de refuerzo *off-policy*. 11, 49, 50, 52, 70
- reward shaping** Diseño de recompensas intermedias que guían la exploración del agente, para compensar la escasez de señal en tareas de horizonte largo. 10, 46, 70
- solver** Programa que resuelve problemas de satisfacción de restricciones o de planificación lógica a partir de un estado y un conjunto de operadores. 7, 70
- sprint** En el marco ágil *Scrum*, ciclo corto y acotado en el tiempo (en este trabajo, la cota es de una semana) en el que el equipo completa una cantidad de trabajo determinada. 12, 70
- stack** En *Minecraft*, conjunto de objetos del mismo tipo agrupados en una única ranura del inventario (habitualmente hasta 64 unidades). 21, 70
- token** Unidad mínima en la que se divide una entrada (texto o parches de imagen) para ser procesada por arquitecturas como los *Transformers*. 37, 70
- woodcutting** Tarea estándar en los entornos de experimentación de *Minecraft*, centrada en la tala y recolección de madera. 10, 13, 20, 24, 25, 29–31, 44, 68, 70
- world model** Modelo que aprende a simular la dinámica de un entorno, de modo que el agente puede predecir estados futuros a partir de sus acciones. 6, 70

- aprendizaje automático** Rama de la inteligencia artificial cuyos algoritmos aprenden a partir de datos, sin programarse de forma explícita para cada tarea. 10, 29, 70
- aprendizaje supervisado** Paradigma de aprendizaje automático en el que el modelo se entrena con datos etiquetados, aprendiendo a asociar cada observación con su salida correspondiente. 9, 70
- bioma** Región de un mundo de *Minecraft* con características propias de clima, vegetación y relieve. 17, 20, 21, 25, 30, 50, 70
- función de recompensa** Función que asigna un valor numérico (recompensa o penalización) a cada transición del agente y define, de forma implícita, el objetivo de la tarea. 46, 70
- LSTM** Unidad de memoria recurrente que permite a una red aprender dependencias a largo plazo en secuencias, mitigando el desvanecimiento del gradiente. 34–36, 48, 70
- línea base** Implementación de referencia con la que se comparan el resto de técnicas. En este trabajo, el agente simbólico (HTN) actúa como línea base interpretable. En inglés, *baseline*. 53, 55, 60, 70
- navegación** Capacidad de un agente para desplazarse de forma autónoma por un entorno tridimensional, planificando rutas y esquivando obstáculos para alcanzar un objetivo. 29, 70
- off-policy** Algoritmo de aprendizaje por refuerzo que aprende sobre una política distinta de la que genera los datos, lo que permite reutilizar transiciones pasadas (por ejemplo, con un *replay buffer*). Se contrapone a los métodos *on-policy*. 11, 49, 50, 70
- on-policy** Algoritmo de aprendizaje por refuerzo que aprende sobre la misma política que genera los datos, por lo que no reutiliza transiciones antiguas. Se contrapone a los métodos *off-policy*; PPO es un ejemplo. 11, 49, 70
- planificador** Algoritmo que busca la secuencia de acciones, o la jerarquía de tareas, que lleva del estado inicial a un estado objetivo. 8, 70
- política** Función que asigna a cada estado del entorno una acción o una distribución de probabilidad sobre las acciones. Puede ser determinista, si devuelve una única acción, o estocástica, si devuelve una distribución. 5, 8–11, 13, 31, 44, 48–50, 55, 56, 70

semilla Valor numérico que sirve de punto de partida a un generador pseudoaleatorio. Se usa para generar mapas idénticos y garantizar la reproducibilidad de los entornos. [17](#), [18](#), [23](#), [50](#), [58](#), [60](#), [70](#)

terminación Final real de un episodio, ya sea porque el agente alcanza el objetivo o porque falla de forma irrecuperable (por ejemplo, al morir). [45](#), [70](#)

truncamiento Final de un episodio forzado por una restricción externa (habitualmente, superar el límite máximo de pasos) y no porque la tarea se complete o falle. [45](#), [47](#), [70](#)

Bibliografía

- [1] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. P. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis, “Mastering the game of go with deep neural networks and tree search,” *Nat.*, vol. 529, no. 7587, pp. 484–489, 2016. [En línea]. Disponible en: <https://doi.org/10.1038/nature16961>
- [2] O. Vinyals, I. Babuschkin, W. M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D. H. Choi, R. Powell, T. Ewalds, P. Georgiev, J. Oh, D. Horgan, M. Kroiss, I. Danihelka, A. Huang, L. Sifre, T. Cai, J. P. Agapiou, M. Jaderberg, A. S. Vezhnevets, R. Leblond, T. Pohlen, V. Dalibard, D. Budden, Y. Sulsky, J. Molloy, T. L. Paine, Ç. Gülçehre, Z. Wang, T. Pfaff, Y. Wu, R. Ring, D. Yogatama, D. Wünsch, K. McKinney, O. Smith, T. Schaul, T. P. Lillicrap, K. Kavukcuoglu, D. Hassabis, C. Apps, and D. Silver, “Grandmaster level in starcraft II using multi-agent reinforcement learning,” *Nat.*, vol. 575, no. 7782, pp. 350–354, 2019. [En línea]. Disponible en: <https://doi.org/10.1038/s41586-019-1724-z>
- [3] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. A. Riedmiller, A. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, “Human-level control through deep reinforcement learning,” *Nat.*, vol. 518, no. 7540, pp. 529–533, 2015. [En línea]. Disponible en: <https://doi.org/10.1038/nature14236>
- [4] M. Kempka, M. Wydmuch, G. Runc, J. Toczec, and W. Jaśkowski, “ViZDoom: A Doom-based AI research platform for visual reinforcement learning,” in *Proceedings of the 2016 IEEE Conference on Computational Intelligence and Games (CIG)*. IEEE, 2016, pp. 1–8.
- [5] W. H. Guss, B. Houghton, N. Topin, P. Wang, C. R. Codel, M. Veloso, and R. Salakhutdinov, “Minerl: A large-scale dataset of minecraft demonstrations,” in *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence*,

- IJCAI 2019, Macao, China, August 10-16, 2019*, S. Kraus, Ed. ijcai.org, 2019, pp. 2442–2448. [En línea]. Disponible en: <https://doi.org/10.24963/ijcai.2019/339>
- [6] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. P. Lillicrap, K. Simonyan, and D. Hassabis, “Mastering chess and shogi by self-play with a general reinforcement learning algorithm,” *CoRR*, vol. abs/1712.01815, 2017. [En línea]. Disponible en: <http://arxiv.org/abs/1712.01815>
- [7] C. Berner, G. Brockman, B. Chan, V. Cheung, P. Debiak, C. Dennison, D. Farhi, Q. Fischer, S. Hashme, C. Hesse, R. Józefowicz, S. Gray, C. Olsson, J. Pachocki, M. Petrov, H. P. de Oliveira Pinto, J. Raiman, T. Salimans, J. Schlatter, J. Schneider, S. Sidor, I. Sutskever, J. Tang, F. Wolski, and S. Zhang, “Dota 2 with large scale deep reinforcement learning,” *CoRR*, vol. abs/1912.06680, 2019. [En línea]. Disponible en: <http://arxiv.org/abs/1912.06680>
- [8] S. Ontañón and M. Buro, “Adversarial hierarchical-task network planning for complex real-time games,” in *Proceedings of the Twenty-Fourth International Joint Conference on Artificial Intelligence, IJCAI 2015, Buenos Aires, Argentina, July 25-31, 2015*, Q. Yang and M. J. Wooldridge, Eds. AAAI Press, 2015, pp. 1652–1658. [En línea]. Disponible en: <http://ijcai.org/Abstract/15/236>
- [9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, I. Guyon, U. von Luxburg, S. Bengio, H. M. Wallach, R. Fergus, S. V. N. Vishwanathan, and R. Garnett, Eds., 2017, pp. 5998–6008. [En línea]. Disponible en: <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>
- [10] B. Baker, I. Akkaya, P. Zhokhov, J. Huizinga, J. Tang, A. Ecoffet, B. Houghton, R. Sampedro, and J. Clune, “Video pretraining (VPT): learning to act by watching unlabeled online videos,” in *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., 2022. [En línea]. Disponible en: http://papers.nips.cc/paper_files/paper/2022/hash/9c7008aff45b5d8f0973b23e1a22ada0-Abstract-Conference.html
- [11] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, “Voyager: An open-ended embodied agent with large language models,” *CoRR*, vol. abs/2305.16291, 2023. [En línea]. Disponible en: <https://doi.org/10.48550/arXiv.2305.16291>

- [12] L. Fan, G. Wang, Y. Jiang, A. Mandlekar, Y. Yang, H. Zhu, A. Tang, D. Huang, Y. Zhu, and A. Anandkumar, “Minedojo: Building open-ended embodied agents with internet-scale knowledge,” in *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., 2022. [En línea]. Disponible en: http://papers.nips.cc/paper_files/paper/2022/hash/74a67268c5cc5910f64938cac4526a90-Abstract-Datasets_and_Benchmarks.html
- [13] Decart, J. Quevedo, Q. McIntyre, S. Campbell, X. Chen, and R. Wachen, “Oasis: A universe in a transformer,” 2024. [En línea]. Disponible en: <https://oasis-model.github.io/>
- [14] I. White*, K. Nottingham*, A. Maniar, M. Robinson, H. Lillemark, M. Maheshwari, L. Qin, and P. Ammanabrolu, “Collaborating action by action: A multi-agent llm framework for embodied reasoning,” *arXiv preprint arXiv:2504.17950*, 2025. [En línea]. Disponible en: <https://arxiv.org/abs/2504.17950>
- [15] S. Ross, G. J. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, AISTATS 2011, Fort Lauderdale, USA, April 11-13, 2011*, ser. JMLR Proceedings, G. J. Gordon, D. B. Dunson, and M. Dudík, Eds. JMLR.org, 2011, pp. 627–635. [En línea]. Disponible en: <http://proceedings.mlr.press/v15/ross11a/ross11a.pdf>
- [16] D. Hafner, J. Pasukonis, J. Ba, and T. P. Lillicrap, “Mastering diverse domains through world models,” *CoRR*, vol. abs/2301.04104, 2023. [En línea]. Disponible en: <https://doi.org/10.48550/arXiv.2301.04104>
- [17] C. Scheller, Y. Schraner, and M. Vogel, “Sample efficient reinforcement learning through learning from demonstrations in minecraft,” *CoRR*, vol. abs/2003.06066, 2020. [En línea]. Disponible en: <https://arxiv.org/abs/2003.06066>
- [18] R. Fikes and N. J. Nilsson, “STRIPS: A new approach to the application of theorem proving to problem solving,” *Artif. Intell.*, vol. 2, no. 3/4, pp. 189–208, 1971. [En línea]. Disponible en: [https://doi.org/10.1016/0004-3702\(71\)90010-5](https://doi.org/10.1016/0004-3702(71)90010-5)
- [19] H. A. Kautz and B. Selman, “Planning as satisfiability,” in *10th European Conference on Artificial Intelligence, ECAI 92, Vienna, Austria, August 3-7, 1992. Proceedings*, B. Neumann, Ed. John Wiley and Sons, 1992, pp. 359–363.
- [20] K. Erol, J. A. Hendler, and D. S. Nau, “UMCP: A sound and complete procedure for hierarchical task-network planning,” in *Proceedings of the Second International*

- Conference on Artificial Intelligence Planning Systems, University of Chicago, Chicago, Illinois, USA, June 13-15, 1994*, K. J. Hammond, Ed. AAAI, 1994, pp. 249–254. [En línea]. Disponible en: <http://www.aaai.org/Library/AIPS/1994/aips94-042.php>
- [21] M. Zare, P. M. Kebria, A. Khosravi, and S. Nahavandi, “A survey of imitation learning: Algorithms, recent developments, and challenges,” *IEEE Trans. Cybern.*, vol. 54, no. 12, pp. 7173–7186, 2024. [En línea]. Disponible en: <https://doi.org/10.1109/TCYB.2024.3395626>
- [22] R. S. Sutton and A. G. Barto, *Reinforcement learning - an introduction, 2nd Edition*. MIT Press, 2018. [En línea]. Disponible en: <http://www.incompleteideas.net/book/the-book-2nd.html>
- [23] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*, ser. Wiley Series in Probability and Statistics. Wiley, 1994. [En línea]. Disponible en: <https://doi.org/10.1002/9780470316887>
- [24] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *CoRR*, vol. abs/1707.06347, 2017. [En línea]. Disponible en: <http://arxiv.org/abs/1707.06347>
- [25] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” *CoRR*, vol. abs/1801.01290, 2018. [En línea]. Disponible en: <http://arxiv.org/abs/1801.01290>
- [26] S. Levine, A. Kumar, G. Tucker, and J. Fu, “Offline reinforcement learning: Tutorial, review, and perspectives on open problems,” *CoRR*, vol. abs/2005.01643, 2020. [En línea]. Disponible en: <https://arxiv.org/abs/2005.01643>
- [27] K. Schwaber and J. Sutherland, *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*, 2020, available at Scrum Guides. [En línea]. Disponible en: <https://scrumguides.org>
- [28] S. Rhein and contributors, “Mineflayer: Create minecraft bots with a node.js api,” n.d. [En línea]. Disponible en: <https://github.com/PrismarineJS/mineflayer>
- [29] J. Redmon, S. K. Divvala, R. B. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*. IEEE Computer Society, 2016, pp. 779–788. [En línea]. Disponible en: <https://doi.org/10.1109/CVPR.2016.91>
- [30] minerllabs, “problem with gym while installing minerl (issue #788),” GitHub issue, <https://github.com/minerllabs/minerl/issues/788>, 2024, consultado el 13 de julio de 2026.

- [31] PaperMC, “Paper: High performance minecraft server software,” n.d. [En línea]. Disponible en: <https://papermc.io>
- [32] OpenJS Foundation, “Node.js: Javascript runtime built on chrome’s v8 javascript engine,” n.d. [En línea]. Disponible en: <https://nodejs.org>
- [33] Google and contributors, “Puppeteer: Javascript api for chrome and firefox,” n.d. [En línea]. Disponible en: <https://github.com/puppeteer/puppeteer>
- [34] Y. LeCun, L. Bottou, G. B. Orr, and K. Müller, “Efficient backprop,” in *Neural Networks: Tricks of the Trade - Second Edition*, ser. Lecture Notes in Computer Science, G. Montavon, G. B. Orr, and K. Müller, Eds. Springer, 2012, vol. 7700, pp. 9–48. [En línea]. Disponible en: https://doi.org/10.1007/978-3-642-35289-8_3
- [35] J. Deng, W. Dong, R. Socher, L. Li, K. Li, and L. Fei-Fei, “Imagenet: A large-scale hierarchical image database,” in *2009 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR 2009), 20-25 June 2009, Miami, Florida, USA*. IEEE Computer Society, 2009, pp. 248–255. [En línea]. Disponible en: <https://doi.org/10.1109/CVPR.2009.5206848>
- [36] Y. Wu and K. He, “Group normalization,” *Int. J. Comput. Vis.*, vol. 128, no. 3, pp. 742–755, 2020. [En línea]. Disponible en: <https://doi.org/10.1007/s11263-019-01198-w>
- [37] H. van Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double q-learning,” in *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence, February 12-17, 2016, Phoenix, Arizona, USA*, D. Schuurmans and M. P. Wellman, Eds. AAAI Press, 2016, pp. 2094–2100. [En línea]. Disponible en: <https://doi.org/10.1609/aaai.v30i1.10295>